GUÍA DE CÓDIGO · BUENA ENERGÍA

App.vue, explicado de forma sencilla

Documentación funcional del componente principal de la tienda, con capturas PNG de fragmentos reales y sus números de línea.

Archivo analizado: **src/App.vue**

La guía corresponde al contenido presente al momento de generarla.

1. Qué es App.vue

App.vue es el componente principal de la tienda. En un único archivo Vue reúne la lógica de la página y la estructura visual que se renderiza en el navegador.

El bloque <script setup> define datos, cálculos y acciones. El bloque <template> describe lo que ve el usuario y conecta los controles con esas acciones. Los estilos se encuentran aparte, en src/style.css; las solicitudes HTTP se delegan a src/services/api.js.

La pantalla combina el catálogo público con paneles que se muestran solo si la sesión indica que la persona es administradora.

2. Datos y reactividad

ref() crea valores reactivos: cuando cambian, Vue vuelve a dibujar las partes de la pantalla que dependen de ellos. Por ejemplo, searchQuery guarda el texto de búsqueda y cartItems guarda las unidades del carrito.

Los grupos de estado principales son: catálogo y filtros (líneas 18–25, 67–69); producto y carrito (25–34); checkout (35–44); autenticación (45–49 y 112–120); administración y productos (50–74).

computed() crea resultados derivados sin duplicar datos. Los filtros y totales se recalculan cuando cambia el catálogo o el estado relacionado. No hacen solicitudes por sí mismos.

3. Funciones comunes y catálogo

La validación de formularios convierte los errores nativos del navegador en mensajes en español (76–110). readStoredUser() recupera la sesión almacenada, y emptyProductForm() entrega un formulario nuevo sin datos anteriores (123–141).

normalizeSearchText() quita diferencias de mayúsculas y acentos. filteredProducts busca por nombre, descripción, categoría, talle y color; cada palabra escrita debe coincidir con algún dato del producto. El inventario y el selector de edición tienen sus propios filtros.

imageForVariant() decide qué foto mostrar en este orden: imagen de la variante, imagen del color, imagen general del producto y, por último, una imagen de respaldo. loadCatalog() trae los productos, actualiza el carrito guardado y corrige selecciones agotadas.

4. Carrito y compra

addToCart() comprueba que la variante exista y tenga stock. Si ya estaba en el carrito, no deja superar las unidades disponibles. Si es nueva, guarda producto, variante, precio, stock e imagen correcta.

El carrito se guarda en localStorage para persistir al recargar la página. Los cambios de cantidad y la eliminación se guardan en el mismo sitio. El subtotal y el contador son computed.

El checkout pide un CAPTCHA al abrirse y envía al backend solo el identificador de variante y la cantidad. El backend confirma precios y disponibilidad. Si registra el pedido, App.vue prepara el mensaje de WhatsApp y vacía el carrito; si falla, muestra el error y conserva la compra.

5. Administración

submitAuth() utiliza los servicios de inicio de sesión o registro. La respuesta se almacena en localStorage y permite que isAdministrator muestre el panel. logout() elimina los datos locales de sesión.

El panel presenta pedidos, productos e inventario. Los pedidos se cargan desde el backend; confirmar, cancelar o entregar cambia su estado. Cuando corresponde, se vuelve a cargar el catálogo para reflejar el stock.

editProduct() copia los datos existentes al formulario, desplaza la pantalla hasta él y enfoca el nombre. submitProduct() distingue entre crear y actualizar. Las imágenes se pueden cargar desde un archivo permitido o indicar mediante URL.

saveVariantStock() solo envía números enteros no negativos. El inventario agrupa variantes por producto y muestra si están disponibles, tienen poco stock o están agotadas.

6. Estructura visual del template

La cabecera ofrece enlaces a la tienda y botones de administración y carrito. El panel de administración se monta condicionalmente; sus pestañas alternan pedidos, edición de productos e inventario.

El catálogo ofrece búsqueda, categorías, tarjetas de producto y mensajes para carga, error y cero resultados. Las secciones de marca y contacto cierran la parte pública.

Las ventanas emergentes contienen detalle de producto, acceso de administrador, bolsa y checkout. Transition agrega animación. Los eventos @click y @submit conectan botones y formularios con las funciones del script; v-model sincroniza controles con refs.

El aviso temporal y el botón flotante de bolsa solo aparecen cuando el carrito está disponible y no hay otra ventana que lo cubra.

7. Recorrido de los datos

Al abrir la aplicación, onMounted() recupera el carrito guardado y llama a loadCatalog(). Las respuestas se guardan en refs; computed prepara los resultados y el template los presenta.

Cuando el usuario cambia un filtro, una variante o una cantidad, v-model actualiza el estado. Las funciones de acción validan la operación y, cuando corresponde, llaman a api.js. Un error se muestra en pantalla en vez de simular un resultado exitoso.

El frontend no es la autoridad para precios o stock del pedido: estos se verifican en el servidor al crear el pedido. La limpieza de temporizadores y vista previa de imágenes ocurre en onUnmounted().

8. Índice de funciones y rangos

El índice resume dónde encontrar cada responsabilidad. Los números corresponden a la versión de App.vue documentada en este PDF; pueden cambiar si se edita el archivo.

Tema	Líneas
Estado y formularios	1–141
Categorías, filtros y búsqueda	142–220
Formato, imágenes y carga del catálogo	222–302
Carrito	303–417
Checkout y WhatsApp	418–526
Autenticación y panel	527–601
Edición de productos y variantes	582–776
Stock y ciclo de vida	777–824
Template: panel administrativo	865–1198
Template: portada y catálogo	1200–1330
Template: secciones y ventanas	1331–1585

9. Capturas del código

Las imágenes siguientes son recortes del archivo fuente con numeración de líneas. Sirven para ubicar cada explicación dentro de App.vue; el código no se modifica al crear esta documentación.

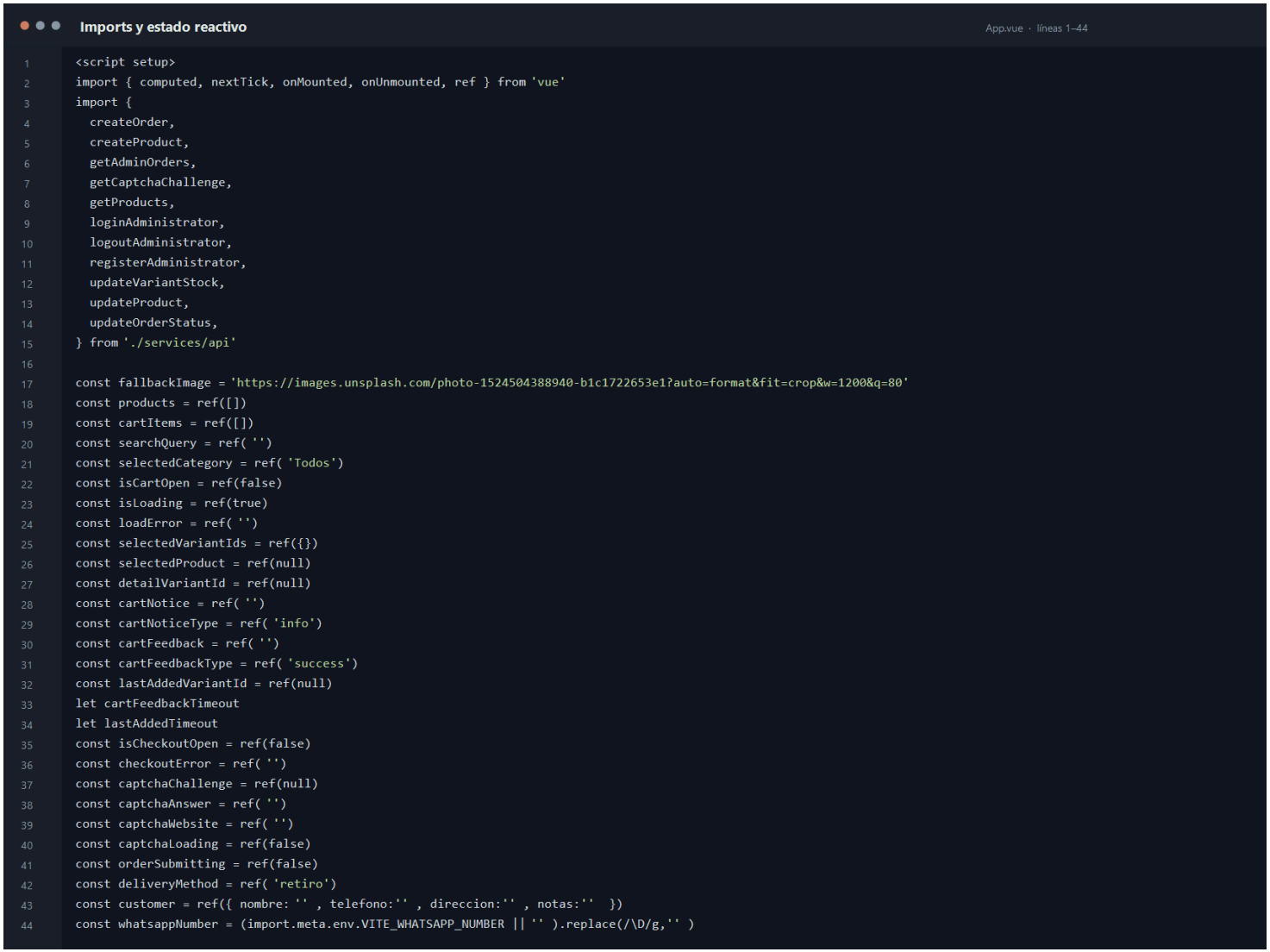
Para encontrar una línea en VS Code, abre src/App.vue y usa **Ctrl+G**, escribe el número y presiona Enter.

Imports y estado reactivo

Líneas 1–44

Dependencias y variables que guardan el estado de la tienda, el carrito, el checkout y la administración.

Que observar: Este bloque conecta el estado del componente con una responsabilidad concreta de la tienda. Seguir el dato desde su origen hasta la accion ayuda a ver como una interaccion termina reflejada en la interfaz.



Mensajes de validación

Líneas 76–110

Conversión de errores nativos del navegador a mensajes claros en español.

Que observar: Centralizar esta regla evita repetirla en varios formularios y mantiene un comportamiento coherente. La plantilla presenta el resultado, y las funciones de accion impiden continuar si los datos no son validos.

Mensajes de validaciÃ³n — App.vue · líneas 76–110

```
76  const setSpanishValidationMessage = (event) => {
77    const field = event.target
78
79    if (!(field instanceof HTMLInputElement || field instanceof HTMLSelectElement || field instanceof HTMLTextAreaElement)) return
80
81    const { validity } = field
82
83    if (validity.valueMissing) {
84      field.setCustomValidity(field.type === 'checkbox'  ?'Marca esta opciÃ³n para continuar.'  :'Completa este campo.' )
85    } else if (validity.typeMismatch && field.type === 'email' ) {
86      field.setCustomValidity( 'Ingresa un correo electrÃ³nico vÃ¡lido.')
87    } else if (validity.typeMismatch && field.type === 'url' ) {
88      field.setCustomValidity( 'Ingresa una URL vÃ¡lida.')
89    } else if (validity.tooShort) {
90      field.setCustomValidity( `Ingresa al menos ${field.minLength} caracteres.`)
91    } else if (validity.tooLong) {
92      field.setCustomValidity( `Usa como mÃ¡ximo ${field.maxLength} caracteres.`)
93    } else if (validity.rangeUnderflow) {
94      field.setCustomValidity( `El valor debe ser como mÃnimo ${field.min}.`)
95    } else if (validity.rangeOverflow) {
96      field.setCustomValidity( `El valor debe ser como mÃ¡ximo ${field.max}.`)
97    } else if (validity.stepMismatch || validity.badInput) {
98      field.setCustomValidity( 'Ingresa un valor vÃ¡lido.')
99    } else if (validity.patternMismatch) {
100     field.setCustomValidity( 'Revisa el formato ingresado.')
101   } else {
102     field.setCustomValidity( 'Revisa el valor ingresado.')
103   }
104 }
105
106 const clearSpanishValidationMessage = (event) => {
107   if (typeof event.target?.setCustomValidity === 'function' ) {
108     event.target.setCustomValidity( '')
109   }
110 }
```

Valores iniciales de formularios

Líneas 112–141

Datos iniciales de autenticación y creación de producto, más lectura segura del usuario guardado.

Que observar: Centralizar esta regla evita repetirla en varios formularios y mantiene un comportamiento coherente. La plantilla presenta el resultado, y las funciones de accion impiden continuar si los datos no son validos.

Valores iniciales de formularios — App.vue · líneas 112–141

```
112  const authForm = ref({
113    nombre: '' ,
114    email: '' ,
115    password: '' ,
116    password_confirmation: '' ,
117    registration_key: '' ,
118  })
119
120  const productForm = ref(emptyProductForm())
121  const isAdministrator = computed(() => currentUser.value?.tipo === 'administrador' )
122
123  function readStoredUser() {
124    try {
125      return JSON.parse(localStorage.getItem( 'user') ||'null' )
126    } catch {
127      return null
128    }
129  }
130
131  function emptyProductForm() {
132    return {
133      nombre: '' ,
134      descripcion: '' ,
135      precio: '' ,
136      categoria: '' ,
137      imagen: '' ,
138      variantes: [{ talle: '' , color:'' , stock: 0 }],
139    }
140  }
141
```

Filtro del inventario

Líneas 142–177

Opciones de categoría y filtrado del inventario por nombre, talle, color y stock.

Que observar: El inventario se cambia para una variante y no desde una cifra visual aislada. El frontend valida el valor antes de enviarlo; la respuesta del servidor determina si el cambio quedo guardado.

Filtro del inventario — App.vue · líneas 142–177

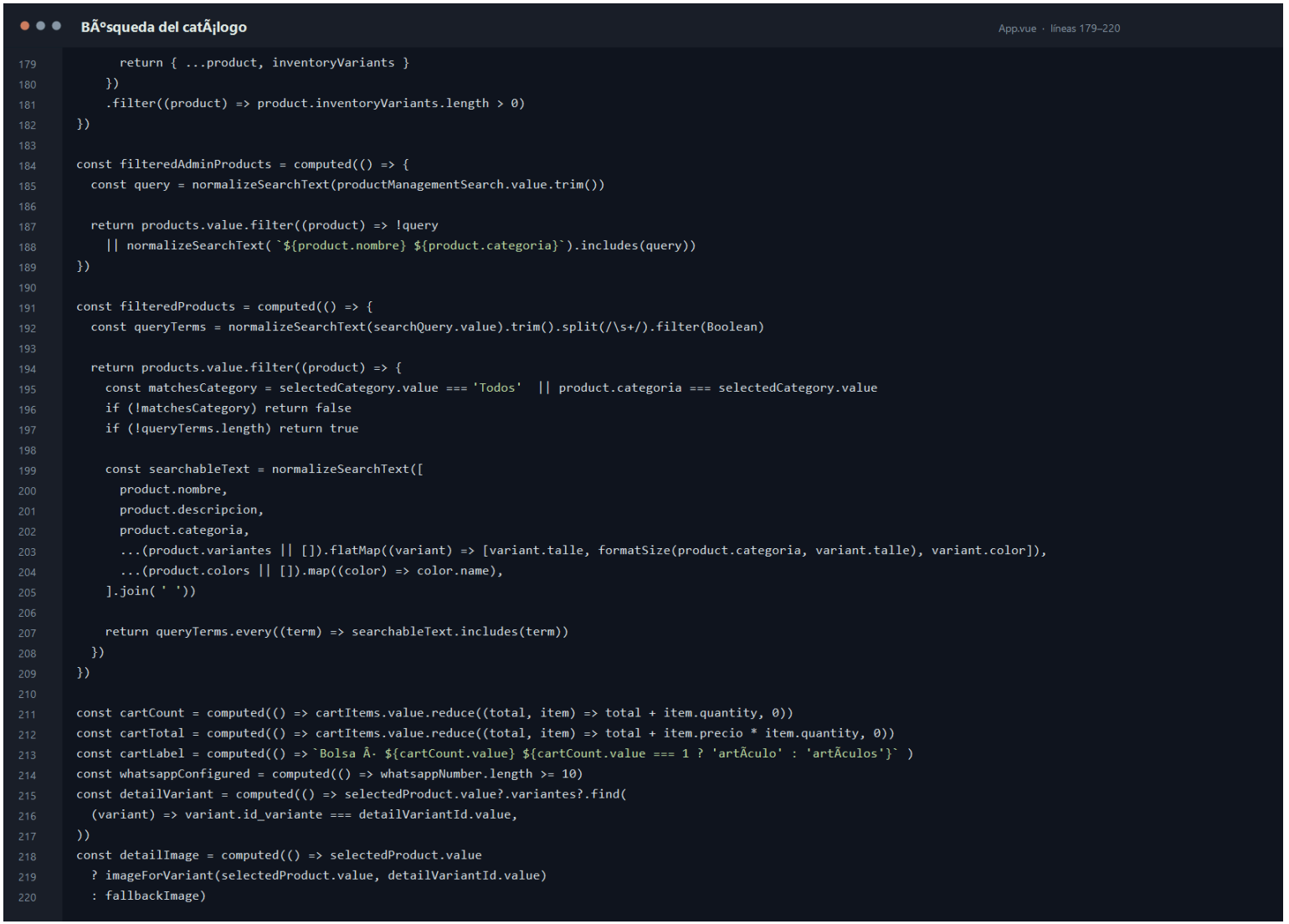
```
142  const normalizeSearchText = (value) => String(value ?? '' )
143    .normalize( 'NFD')
144    .replace(/[\u0300-\u036f]/g, '' )
145    .toLocaleLowerCase( 'es')
146
147  const categoryOptions = computed(() => {
148    const categories = [...new Set(products.value.map((product) => product.categoria).filter(Boolean))]
149      .sort((firstCategory, secondCategory) => firstCategory.localeCompare(secondCategory, 'es' ))
150
151    return [
152      { name: 'Todos' , count: products.value.length },
153      ...categories.map((name) => ({
154        name,
155        count: products.value.filter((product) => product.categoria === name).length,
156      })),
157    ]
158  })
159
160  const pendingOrderCount = computed(() => orders.value.filter((order) => order.estado === 'Pendiente' ).length)
161  const filteredInventoryProducts = computed(() => {
162    const query = normalizeSearchText(inventorySearch.value.trim())
163
164    return products.value
165      .map((product) => {
166        const productMatches = normalizeSearchText(product.nombre).includes(query)
167        const inventoryVariants = (product.variantes || []).filter((variant) => {
168          const stock = Number(stockDrafts.value[variant.id_variante] ?? variant.stock)
169          const matchesFilter = inventoryFilter.value === 'todos'
170            || (inventoryFilter.value === 'poco'  && stock > 0 && stock <= 5)
171            || (inventoryFilter.value === 'agotado'  && stock === 0)
172          const matchesSearch = !query
173            || productMatches
174            || normalizeSearchText( `${variant.talle} ${variant.color}`).includes(query)
175
176          return matchesFilter && matchesSearch
177        })
```

Búsqueda del catálogo

Líneas 179–220

Búsqueda por palabras en nombre, descripción, categoría, talle y color; también se calculan totales y detalles.

Que observar: El resultado se deriva de los datos y filtros actuales, asi que se recalcula al escribir o cambiar una opcion. Esto mantiene la busqueda inmediata y evita solicitar al servidor por cada caracter.



Formato e imágenes

Líneas 222–255

Formato de moneda y talles, totales de stock y elección de imagen según variante y color.

Que observar: La prioridad de imagen evita que el producto cambie de aspecto al seleccionar una variante. Cuando se carga un archivo, la vista previa sirve para comprobarlo antes de persistir el cambio.

Formato e imÃ¡genes — App.vue · líneas 222–255

```
222  const formatPrice = (price) => new Intl.NumberFormat( 'es-UY', {
223    style: 'currency' ,
224    currency: 'UYU' ,
225    maximumFractionDigits: 0,
226  }).format(Number(price) || 0)
227
228  const formatSize = (category, size) => category === 'Infantil'  ?`${size} aÃ±os`  : size
229  const formatCartSize = (item) => {
230    const category = item.categoria || products.value.find((product) => product.id_producto === item.id_producto)?.categoria
231    return formatSize(category, item.talle)
232  }
233
234  const productStockTotal = (product, variants = product.variantes || []) => variants.reduce(
235    (total, variant) => total + Number(variant.stock),
236    0,
237  )
238
239  const imageForVariant = (product, variantId) => {
240    const variant = product.variantes?.find((item) => item.id_variante === variantId)
241    const color = product.colors?.find((item) => item.name?.toLocaleLowerCase( 'es') === variant?.color?.toLocaleLowerCase('es'))
242    return variant?.imagen_url || color?.image || product.imagen_url || fallbackImage
243  }
244
245  const adminProductImage = (product) => {
246    const selectedVariantId = selectedVariantIds.value[product.id_producto]
247    const representativeVariant = product.variantes?.find((variant) => variant.id_variante === selectedVariantId)
248      || product.variantes?.find((variant) => Number(variant.stock) > 0)
249      || product.variantes?.[0]
250
251    return representativeVariant
252      ? imageForVariant(product, representativeVariant.id_variante)
253      : product.imagen_url || fallbackImage
254  }
255
```

Carga del catálogo

Líneas 256–280

Inicio de carga y actualización de datos de cada artículo persistido en el carrito.

Que observar: El resultado se deriva de los datos y filtros actuales, asi que se recalcula al escribir o cambiar una opcion. Esto mantiene la busqueda inmediata y evita solicitar al servidor por cada caracter.



Sincronización del carrito

Líneas 281–302

Persistencia de imágenes corregidas y selección de variantes disponibles al actualizar el catálogo.

Que observar: El estado del carrito alimenta el contador, el subtotal y la bolsa visible. Guardar los cambios en el navegador permite recuperarlos al volver, mientras la validacion evita superar las existencias conocidas.

SincronizaciÃ³n del carrito — App.vue · líneas 281–302

```
      (product.variantes || []).map((variant) => [variant.id_variante, Number(variant.stock)]),
    ))
    const selections = { ...selectedVariantIds.value }

    for (const product of products.value) {
      const variants = product.variantes || []
      const currentVariant = variants.find((variant) => variant.id_variante === selections[product.id_producto])

      if (!currentVariant || Number(currentVariant.stock) < 1) {
        const availableVariant = variants.find((variant) => Number(variant.stock) > 0)
        if (availableVariant) selections[product.id_producto] = availableVariant.id_variante
      }
    }

    selectedVariantIds.value = selections
  } catch (error) {
    loadError.value = error.message || 'No fue posible cargar el catÃ¡logo.'
  } finally {
    isLoading.value = false
  }
}

```

Avisos del carrito

Líneas 303–331

Guardado local, notificaciones temporales y confirmación visual de un artículo agregado.

Que observar: El estado del carrito alimenta el contador, el subtotal y la bolsa visible. Guardar los cambios en el navegador permite recuperarlos al volver, mientras la validacion evita superar las existencias conocidas.

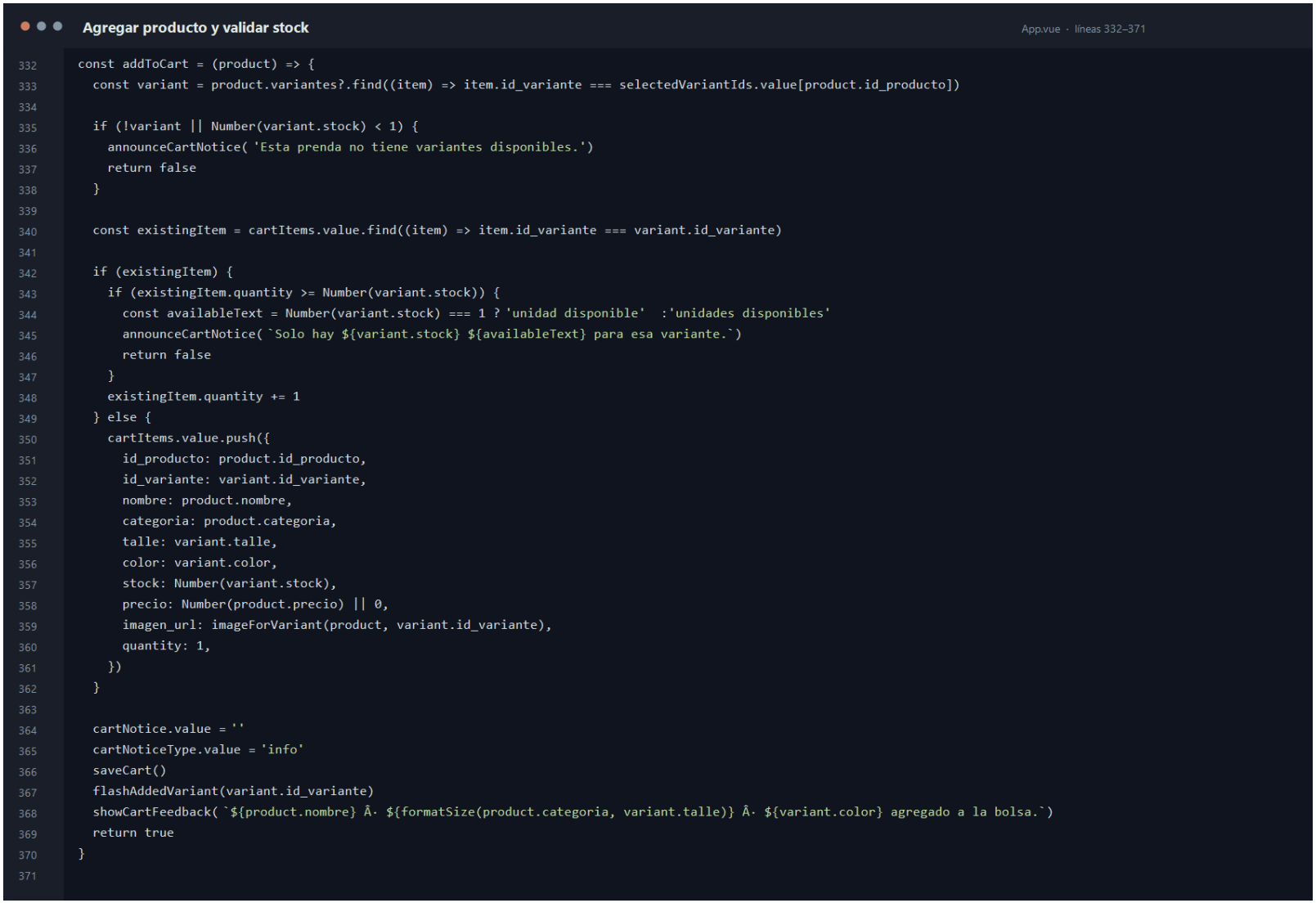


Agregar producto y validar stock

Líneas 332–371

Límites de stock y creación de una línea de carrito con la imagen de su variante.

Que observar: El inventario se cambia para una variante y no desde una cifra visual aislada. El frontend valida el valor antes de enviarlo; la respuesta del servidor determina si el cambio quedo guardado.




```
const addToCart = (product) => {
  const variant = product.variantes?.find((item) => item.id_variante === selectedVariantIds.value[product.id_producto])

  if (!variant || Number(variant.stock) < 1) {
    announceCartNotice( 'Esta prenda no tiene variantes disponibles.')
    return false
  }

  const existingItem = cartItems.value.find((item) => item.id_variante === variant.id_variante)

  if (existingItem) {
    if (existingItem.quantity >= Number(variant.stock)) {
      const availableText = Number(variant.stock) === 1 ? 'unidad disponible'  :'unidades disponibles'
      announceCartNotice( `Solo hay ${variant.stock} ${availableText} para esa variante.`)
      return false
    }
    existingItem.quantity += 1
  } else {
    cartItems.value.push({
      id_producto: product.id_producto,
      id_variante: variant.id_variante,
      nombre: product.nombre,
      categoria: product.categoria,
      talle: variant.talle,
      color: variant.color,
      stock: Number(variant.stock),
      precio: Number(product.precio) || 0,
      imagen_url: imageForVariant(product, variant.id_variante),
      quantity: 1,
    })
  }

  cartNotice.value = ''
  cartNoticeType.value = 'info'
  saveCart()
  flashAddedVariant(variant.id_variante)
  showCartFeedback( `${product.nombre} Â· ${formatSize(product.categoria, variant.talle)} Â· ${variant.color} agregado a la bolsa.`)
  return true
}

```

Detalle y cantidad del producto

Líneas 372–393

Apertura del detalle y selección de la variante que se agrega.

Que observar: Este bloque prepara o modifica datos del producto antes de guardarlos. Las comprobaciones y la respuesta del servicio mantienen el formulario sincronizado con el catalogo real.

Detalle y cantidad del producto — App.vue · líneas 372–393

```
372  const openProductDetails = (product) => {
373    selectedProduct.value = product
374    const selectedId = selectedVariantIds.value[product.id_producto]
375    const selectedVariant = product.variantes?.find((variant) => variant.id_variante === selectedId && Number(variant.stock) > 0)
376    const firstAvailableVariant = product.variantes?.find((variant) => Number(variant.stock) > 0)
377    detailVariantId.value = (selectedVariant || firstAvailableVariant || product.variantes?.[0])?.id_variante || null
378  }
379
380  const addDetailToCart = () => {
381    if (!selectedProduct.value || !detailVariantId.value) return
382
383    selectedVariantIds.value = {
384      ...selectedVariantIds.value,
385      [selectedProduct.value.id_producto]: detailVariantId.value,
386    }
387
388    if (addToCart(selectedProduct.value)) {
389      selectedProduct.value = null
390    }
391  }
392
393  const updateQuantity = (item, change) => {
```

Actualizar cantidad y quitar artículos

Líneas 394–417

Control de stock al modificar cantidades y persistencia de la eliminación.

Que observar: Este bloque conecta el estado del componente con una responsabilidad concreta de la tienda. Seguir el dato desde su origen hasta la accion ayuda a ver como una interaccion termina reflejada en la interfaz.

Actualizar cantidad y quitar artÃculos — App.vue · líneas 394–417

```
394   if (change > 0 && item.quantity >= item.stock) {
395     const availableText = item.stock === 1 ? 'unidad disponible'  :'unidades disponibles'
396     announceCartNotice( `Solo hay ${item.stock} ${availableText} de ${item.nombre} (${formatCartSize(item)}, ${item.color}).`)
397     return
398   }
399
400   item.quantity += change
401
402   if (item.quantity <= 0) {
403     cartItems.value = cartItems.value.filter((cartItem) => cartItem.id_variante !== item.id_variante)
404   }
405
406   cartNotice.value = ''
407   cartNoticeType.value = 'info'
408   saveCart()
409 }
410
411 const removeFromCart = (variantId) => {
412   cartItems.value = cartItems.value.filter((item) => item.id_variante !== variantId)
413   cartNotice.value = ''
414   cartNoticeType.value = 'info'
415   saveCart()
416 }
417
```

Preparar el checkout

Líneas 418–437

Solicitud de un desafío CAPTCHA al abrir el formulario de compra.

Que observar: La compra se prepara y se valida antes de mostrar una confirmacion. Los datos se envian al backend, que vuelve a comprobar stock y precios; solo un resultado correcto permite completar el flujo.

Preparar el checkout — App.vue · líneas 418–437

```
418  const loadCaptchaChallenge = async () => {
419    captchaChallenge.value = null
420    captchaAnswer.value = ''
421    captchaLoading.value = true
422
423    try {
424      captchaChallenge.value = await getCaptchaChallenge()
425    } catch {
426      checkoutError.value = 'No se pudo cargar la verificaciÃ³n. IntÃ©ntalo de nuevo.'
427    } finally {
428      captchaLoading.value = false
429    }
430  }
431
432  const openCheckout = () => {
433    checkoutError.value = ''
434    isCheckoutOpen.value = true
435    void loadCaptchaChallenge()
436  }
437
```

Registrar pedido

Líneas 438–465

Validaciones previas y datos enviados al backend para crear un pedido.

Que observar: Este paso forma parte del ciclo de compra o de su administracion. El servidor es quien confirma el resultado; despues, la interfaz actualiza pedidos y catalogo para no mostrar datos desfasados.

Registrar pedido — App.vue · líneas 438–465

```
438  const submitOrder = async () => {
439    checkoutError.value = ''
440
441    if (!whatsappConfigured.value) {
442      checkoutError.value = 'Falta configurar el WhatsApp de la tienda.'
443      return
444    }
445
446    if (!captchaChallenge.value || captchaLoading.value) {
447      checkoutError.value = 'Espera a que cargue la verificaciÃ³n.'
448      return
449    }
450
451    const whatsappWindow = window.open( '','_blank' )
452
453    if (!whatsappWindow) {
454      checkoutError.value = 'Permite las ventanas emergentes para continuar a WhatsApp.'
455      return
456    }
457
458    whatsappWindow.opener = null
459    orderSubmitting.value = true
460    let orderResponse
461
462    try {
463      orderResponse = await createOrder({
464        cliente: {
465          nombre: customer.value.nombre.trim(),
```

Errores del pedido

Líneas 466–482

Tratamiento de errores del servidor y actualización del catálogo ante un conflicto de stock.

Que observar: Este paso forma parte del ciclo de compra o de su administracion. El servidor es quien confirma el resultado; despues, la interfaz actualiza pedidos y catalogo para no mostrar datos desfasados.



Construir mensaje de WhatsApp

Líneas 483–505

Presentación del detalle, entrega y total del pedido en un mensaje legible.

Que observar: La compra se prepara y se valida antes de mostrar una confirmacion. Los datos se envian al backend, que vuelve a comprobar stock y precios; solo un resultado correcto permite completar el flujo.

Construir mensaje de WhatsApp — App.vue · líneas 483–505

```
483       ? Object.values(validationErrors).flat().join( ' ')
484       : error.response?.data?.message || 'No se pudo registrar el pedido.'
485     await loadCaptchaChallenge()
486     if (error.response?.status === 422) await loadCatalog()
487     return
488   } finally {
489     orderSubmitting.value = false
490   }
491
492   const order = orderResponse.order
493   const itemLines = order.detalles.map((item, index) =>
494     `${index + 1}. ${item.nombre} | Talle: ${formatSize(item.categoria, item.talle)} | Color: ${item.color} | Cantidad: ${item.cantidad} | ${formatPri
    ce(Number(item.precio_unitario) * item.cantidad)}`,
495   )
496
497   const deliveryText = order.tipo_entrega === 'retiro'
498     ? 'Retiro en el local'
499     : `Entrega a domicilio\nDirecciÃ³n: ${order.cliente.direccion}\nEnvÃo: por confirmar con la tienda`
500
501   const message = [
502  `*PEDIDO #${order.id_pedido} Â· BUENA ENERGÃ⊡A*`    ,
503  ''    ,
504  `Cliente: ${order.cliente.nombre}`    ,
505  `TelÃ©fono: ${order.cliente.telefono}`    ,
```

Completar compra

Líneas 506–526

Apertura de WhatsApp y limpieza del carrito después de registrar correctamente el pedido.

Que observar: La compra se prepara y se valida antes de mostrar una confirmacion. Los datos se envian al backend, que vuelve a comprobar stock y precios; solo un resultado correcto permite completar el flujo.

Completar compra — App.vue · líneas 506–526

```
506   `Entrega: ${deliveryText}`    ,
507   ''    ,
508   '*PRODUCTOS*'    ,
509      ...itemLines,
510   ''    ,
511   `*Subtotal: ${formatPrice(order.total)}*`    ,
512      ...(order.notas ? [ '',`Notas: ${order.notas}` ] : []),
513   ''    ,
514   'Pedido pendiente de confirmaciÃ³n de disponibilidad y costo de envÃo.'    ,
515    ].join( '\n')
516
517    const whatsappUrl = `https://wa.me/${whatsappNumber}?text=${encodeURIComponent(message)}`
518    whatsappWindow.location.href = whatsappUrl
519    cartItems.value = []
520    saveCart()
521    isCheckoutOpen.value = false
522    isCartOpen.value = false
523    captchaAnswer.value = ''
524    captchaChallenge.value = null
525  }
526
```

Autenticación de administrador

Líneas 527–560

Inicio de sesión y registro, persistiendo la sesión recibida del backend.

Que observar: El estado de sesion determina que funciones administrativas se habilitan. El componente conserva la respuesta de autenticacion para mostrar la interfaz adecuada, mientras las operaciones protegidas se realizan a traves de la API.

AutenticaciÃ³n de administrador — App.vue · líneas 527–560

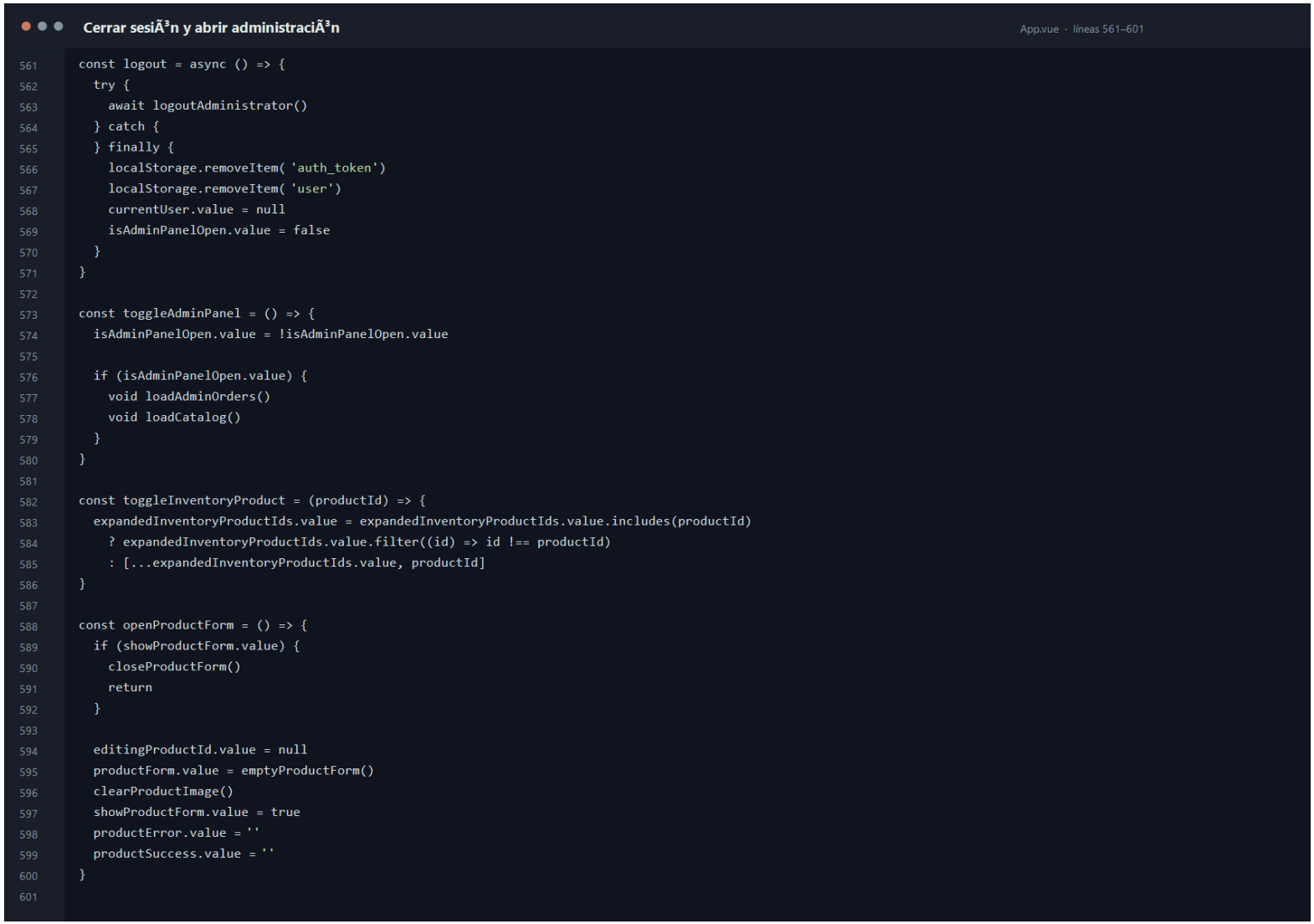
```
527  const handleImageError = (event) => {
528    event.target.src = fallbackImage
529  }
530
531  const openAuth = () => {
532    authMode.value = 'login'
533    authError.value = ''
534    isAuthOpen.value = true
535  }
536
537  const submitAuth = async () => {
538    authLoading.value = true
539    authError.value = ''
540
541    try {
542      const response = authMode.value === 'login'
543        ? await loginAdministrator({ email: authForm.value.email, password: authForm.value.password })
544        : await registerAdministrator(authForm.value)
545
546      localStorage.setItem( 'auth_token', response.token)
547      localStorage.setItem( 'user', JSON.stringify(response.user))
548      currentUser.value = response.user
549      isAuthOpen.value = false
550      authForm.value = { nombre: '' , email:'' , password:'' , password_confirmation:'' , registration_key:''  }
551    } catch (error) {
552      const validationErrors = error.response?.data?.errors
553      authError.value = validationErrors
554        ? Object.values(validationErrors).flat().join( ' ')
555        : error.response?.data?.message || 'No fue posible autenticar al administrador.'
556    } finally {
557      authLoading.value = false
558    }
559  }
560
```

Cerrar sesión y abrir administración

Líneas 561–601

Cierre de sesión, carga de datos administrativos y expansión de productos del inventario.

Que observar: El estado de sesion determina que funciones administrativas se habilitan. El componente conserva la respuesta de autenticacion para mostrar la interfaz adecuada, mientras las operaciones protegidas se realizan a traves de la API.

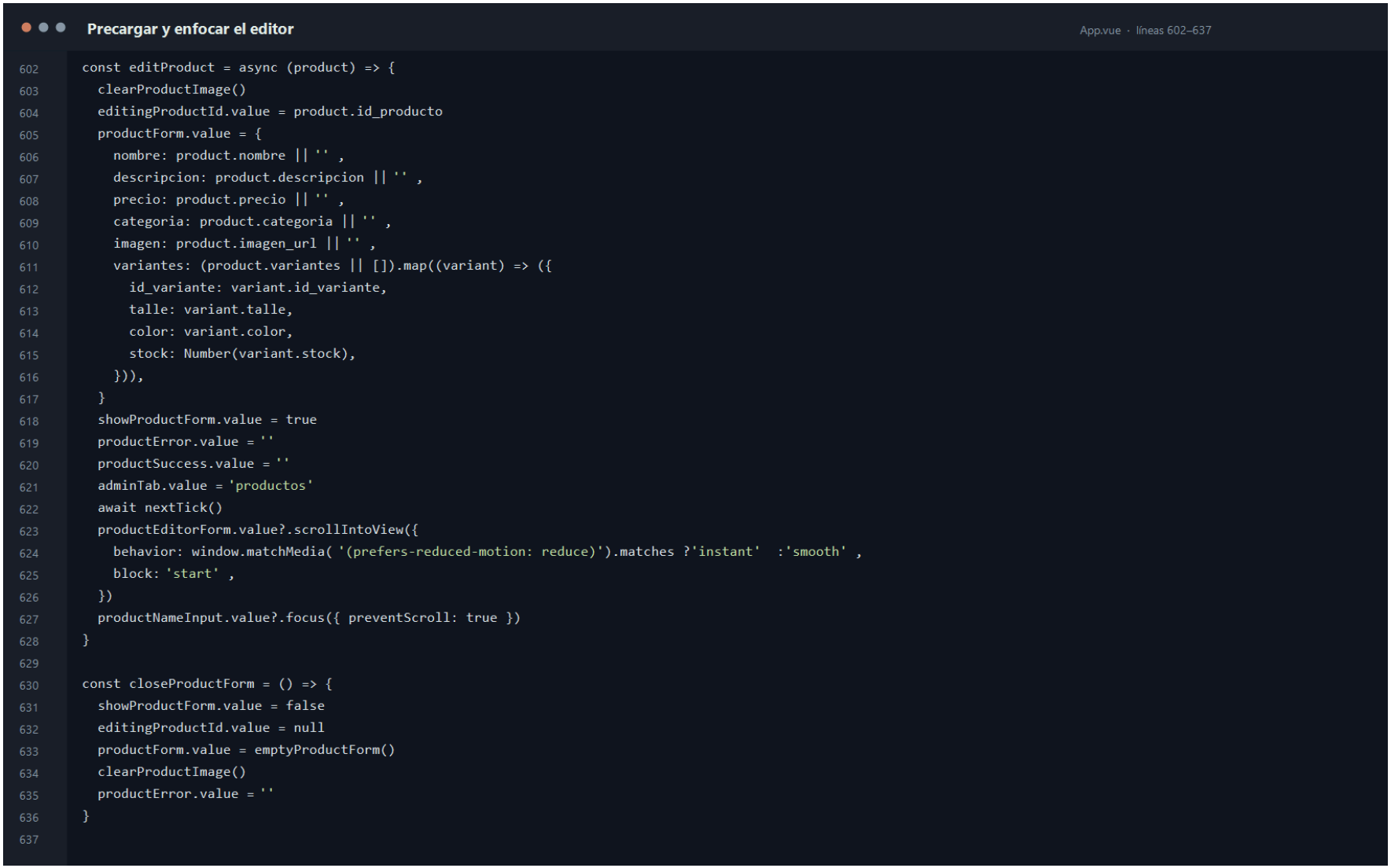


Precargar y enfocar el editor

Líneas 602–637

Precarga de datos y variantes, desplazamiento hasta el formulario y limpieza al cancelar.

Que observar: Este bloque prepara o modifica datos del producto antes de guardarlos. Las comprobaciones y la respuesta del servicio mantienen el formulario sincronizado con el catalogo real.



Cargar pedidos

Líneas 638–652

Carga de pedidos y presentación de un estado de error cuando el servicio falla.

Que observar: Este paso forma parte del ciclo de compra o de su administracion. El servidor es quien confirma el resultado; despues, la interfaz actualiza pedidos y catalogo para no mostrar datos desfasados.



Cambiar estado del pedido

Líneas 653–681

Actualización del pedido y refresco del catálogo cuando el cambio afecta stock.

Que observar: Este paso forma parte del ciclo de compra o de su administracion. El servidor es quien confirma el resultado; despues, la interfaz actualiza pedidos y catalogo para no mostrar datos desfasados.

Cambiar estado del pedido — App.vue · líneas 653–681

```
653  const changeOrderStatus = async (order, status) => {
654    updatingOrderId.value = order.id_pedido
655    ordersError.value = ''
656    ordersSuccess.value = ''
657
658    try {
659      const response = await updateOrderStatus(order.id_pedido, status)
660      order.estado = response.order.estado
661      ordersSuccess.value = `El pedido #${order.id_pedido} quedÃ³ ${order.estado.toLowerCase()}.`
662
663      if (status === 'Confirmado'  || status ==='Cancelado' ) {
664        await loadCatalog()
665      }
666    } catch (error) {
667      const validationErrors = error.response?.data?.errors
668      ordersError.value = validationErrors
669        ? Object.values(validationErrors).flat().join( ' ')
670        : error.response?.data?.message || 'No se pudo actualizar el pedido.'
671      await loadAdminOrders()
672    } finally {
673      updatingOrderId.value = null
674    }
675  }
676
677  const formatOrderDate = (date) => new Intl.DateTimeFormat( 'es-UY', {
678    dateStyle: 'medium' ,
679    timeStyle: 'short' ,
680  }).format(new Date(date))
681
```

Administrar variantes

Líneas 682–706

Agregar y quitar variantes, limpiar vista previa y eliminar la imagen actual del producto.

Que observar: Este bloque prepara o modifica datos del producto antes de guardarlos. Las comprobaciones y la respuesta del servicio mantienen el formulario sincronizado con el catalogo real.

Administrar variantes — App.vue · líneas 682–706

```
682  const addVariant = () => {
683    if (productForm.value.variantes.length < 40) {
684      productForm.value.variantes.push({ talle: '' , color:'' , stock: 0 })
685    }
686  }
687
688  const removeVariant = (index) => {
689    if (productForm.value.variantes.length > 1) {
690      productForm.value.variantes.splice(index, 1)
691    }
692  }
693
694  const clearProductImage = () => {
695    if (productImagePreview.value) URL.revokeObjectURL(productImagePreview.value)
696    productImageFile.value = null
697    productImagePreview.value = ''
698    if (productImageInput.value) productImageInput.value.value = ''
699  }
700
701  const removeProductImage = () => {
702    clearProductImage()
703    productForm.value.imagen = ''
704  }
705
706  const handleProductImageSelection = (event) => {
```

Validar imagen cargada

Líneas 707–729

Tipos de archivo aceptados, límite de tamaño y generación de vista previa.

Que observar: Este bloque prepara o modifica datos del producto antes de guardarlos. Las comprobaciones y la respuesta del servicio mantienen el formulario sincronizado con el catalogo real.

Validar imagen cargada — App.vue · líneas 707–729

```
707   productError.value = ''
708   const file = event.target.files?.[0]
709
710   if (!file) return
711
712   const acceptedTypes = [ 'image/jpeg','image/png' ,'image/webp' ]
713   if (!acceptedTypes.includes(file.type)) {
714     clearProductImage()
715     productError.value = 'Selecciona una imagen JPG, PNG o WebP.'
716     return
717   }
718
719   if (file.size > 5 * 1024 * 1024) {
720     clearProductImage()
721     productError.value = 'La imagen debe pesar mÃ¡ximo 5 MB.'
722     return
723   }
724
725   if (productImagePreview.value) URL.revokeObjectURL(productImagePreview.value)
726   productImageFile.value = file
727   productImagePreview.value = URL.createObjectURL(file)
728 }
729
```

Crear o actualizar producto

Líneas 730–753

Elección entre creación y edición y armado de los datos enviados al servicio.

Que observar: Este bloque prepara o modifica datos del producto antes de guardarlos. Las comprobaciones y la respuesta del servicio mantienen el formulario sincronizado con el catalogo real.

Crear o actualizar producto — App.vue · líneas 730–753

```
730  const submitProduct = async () => {
731    productLoading.value = true
732    productError.value = ''
733    productSuccess.value = ''
734    const isEditing = editingProductId.value !== null
735
736    try {
737      const productData = {
738        ...productForm.value,
739        precio: Number(productForm.value.precio),
740        descripcion: productForm.value.descripcion || null,
741        imagen: productForm.value.imagen || null,
742        imagen_archivo: productImageFile.value,
743        variantes: productForm.value.variantes.map((variant) => ({
744          id_variante: variant.id_variante,
745          talle: variant.talle,
746          color: variant.color,
747          stock: Number(variant.stock),
748        })),
749      }
750
751      if (isEditing) {
752        await updateProduct(editingProductId.value, productData)
753      } else {
```

Resultado al guardar

Líneas 754–776

Mensajes de éxito y error, recarga del catálogo y estado de carga del formulario.

Que observar: Este bloque conecta el estado del componente con una responsabilidad concreta de la tienda. Seguir el dato desde su origen hasta la accion ayuda a ver como una interaccion termina reflejada en la interfaz.

Resultado al guardar — App.vue · líneas 754–776

```
754         await createProduct(productData)
755       }
756
757       productForm.value = emptyProductForm()
758       editingProductId.value = null
759       clearProductImage()
760       showProductForm.value = false
761       productSuccess.value = isEditing
762         ? 'Producto actualizado correctamente.'
763         : 'Producto creado y aÃ±adido al catÃ¡logo.'
764       await loadCatalog()
765     } catch (error) {
766       const validationErrors = error.response?.data?.errors
767       productError.value = validationErrors
768         ? Object.values(validationErrors).flat().join( ' ')
769         : error.response?.data?.message || (isEditing
770           ? 'No se pudo actualizar el producto.'
771           : 'No se pudo crear el producto.' )
772     } finally {
773       productLoading.value = false
774     }
775   }
776
```

Guardar inventario

Líneas 777–804

Validación de unidades y actualización del stock de una variante.

Que observar: El inventario se cambia para una variante y no desde una cifra visual aislada. El frontend valida el valor antes de enviarlo; la respuesta del servidor determina si el cambio quedo guardado.

Guardar inventario — App.vue · líneas 777–804

```
777  const saveVariantStock = async (variant) => {
778    const stock = stockDrafts.value[variant.id_variante]
779
780    if (!Number.isInteger(stock) || stock < 0 || stock > 2147483647) {
781      stockError.value = 'Ingresa una cantidad entera entre 0 y 2147483647.'
782      stockSuccess.value = ''
783      return
784    }
785
786    stockLoading.value = true
787    stockError.value = ''
788    stockSuccess.value = ''
789
790    try {
791      const response = await updateVariantStock(variant.id_variante, stock)
792      variant.stock = Number(response.variant.stock)
793      stockDrafts.value[variant.id_variante] = variant.stock
794      stockSuccess.value = `Stock actualizado para ${variant.talle} Â· ${variant.color}.`
795    } catch (error) {
796      const validationErrors = error.response?.data?.errors
797      stockError.value = validationErrors
798        ? Object.values(validationErrors).flat().join( ' ')
799        : error.response?.data?.message || 'No se pudo actualizar el stock.'
800    } finally {
801      stockLoading.value = false
802    }
803  }
804
```

Ciclo de vida del componente

Líneas 805–824

Restauración del carrito al iniciar y limpieza al desmontar el componente.

Que observar: Estos pasos se ejecutan al entrar o salir de la pagina, no por cada clic. Inicializar datos al montar y liberar recursos al desmontar evita estados perdidos y temporizadores activos innecesariamente.

Ciclo de vida del componente — App.vue · líneas 805–824

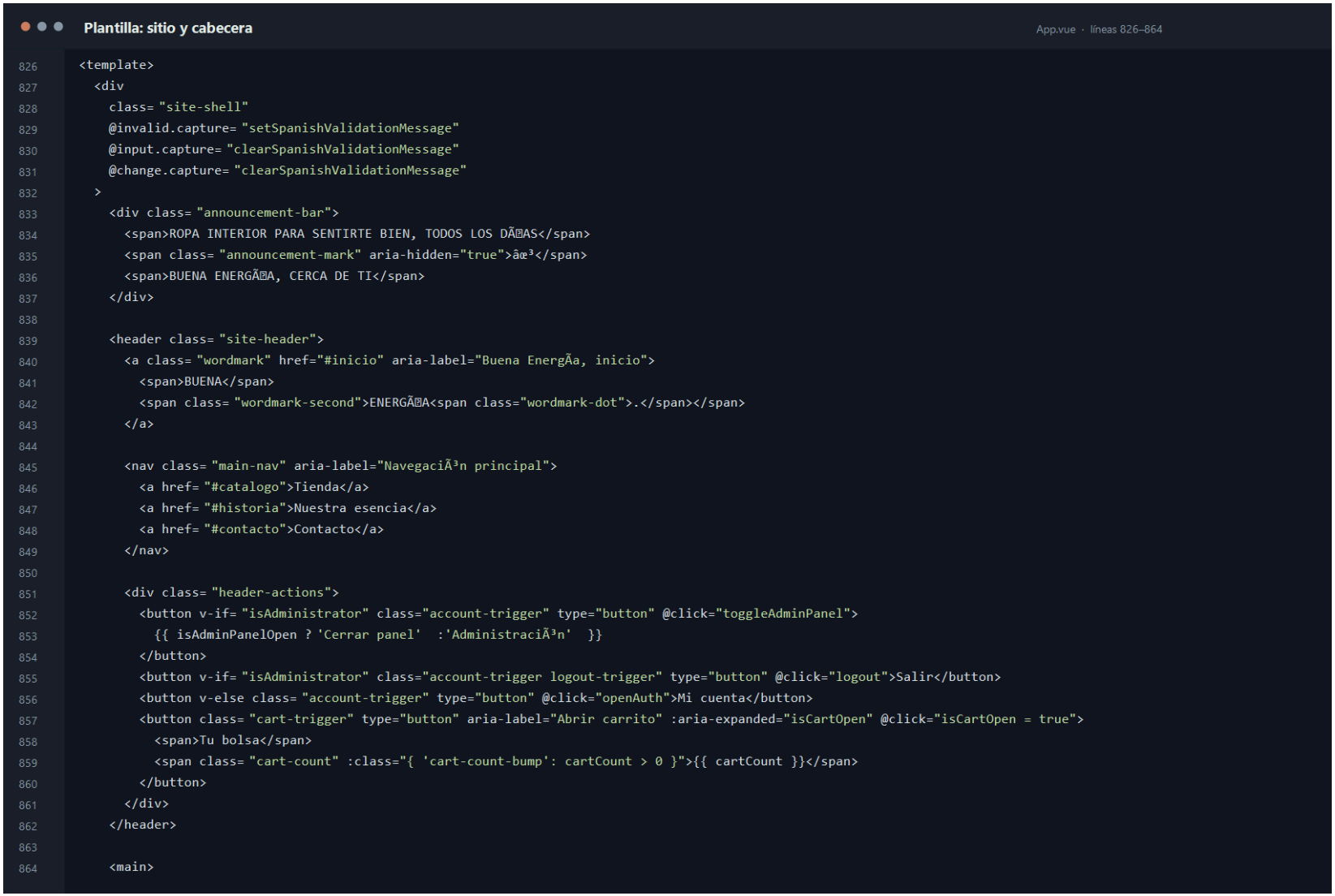
```
805  onMounted(() => {
806    try {
807      const savedCart = JSON.parse(localStorage.getItem( 'buena-energia-cart') ||'[]' )
808      cartItems.value = Array.isArray(savedCart)
809        ? savedCart.filter((item) => item.id_variante && Number(item.quantity) > 0)
810        : []
811      saveCart()
812    } catch {
813      cartItems.value = []
814    }
815
816    loadCatalog()
817  })
818
819  onUnmounted(() => {
820    clearProductImage()
821    if (cartFeedbackTimeout) window.clearTimeout(cartFeedbackTimeout)
822    if (lastAddedTimeout) window.clearTimeout(lastAddedTimeout)
823  })
824  </script>
```

Plantilla: sitio y cabecera

Líneas 826–864

Contenedor global, validación, navegación y accesos de cuenta y carrito.

Que observar: La plantilla decide que mostrar segun el estado actual y conecta botones o campos con las acciones del componente. Esto separa lo que la persona ve de las operaciones que actualizan los datos.



Plantilla: panel y pestañas

Líneas 865–904

El panel solo se muestra al administrador y ofrece secciones de pedidos, inventario y productos.

Que observar: La plantilla decide que mostrar segun el estado actual y conecta botones o campos con las acciones del componente. Esto separa lo que la persona ve de las operaciones que actualizan los datos.

Plantilla: panel y pestaÃ±as — App.vue · líneas 865–904

```
865      <section v-if="isAdministrator && isAdminPanelOpen" class="admin-section" aria-labelledby="admin-title">
866        <div class="admin-heading">
867          <div>
868            <p class="eyebrow"><span class="eyebrow-line"></span> GESTIÃ“N DE TIENDA</p>
869            <h2 id="admin-title">Panel de administraciÃ³n</h2>
870            <p>Pedidos, productos e inventario en un solo lugar.</p>
871          </div>
872          <span class="admin-badge">{{ currentUser.nombre }}</span>
873        </div>
874
875        <nav class="admin-tabs" aria-label="Secciones de administraciÃ³n">
876          <button
877            type="button"
878            :class="{ active: adminTab === 'pedidos' }"
879            :aria-pressed="adminTab === 'pedidos'"
880            aria-controls="admin-orders"
881            @click="adminTab = 'pedidos'"
882          >
883            Pedidos <span v-if="pendingOrderCount" class="admin-tab-count">{{ pendingOrderCount }}</span>
884          </button>
885          <button
886            type="button"
887            :class="{ active: adminTab === 'inventario' }"
888            :aria-pressed="adminTab === 'inventario'"
889            aria-controls="admin-inventory"
890            @click="adminTab = 'inventario'"
891          >
892            Inventario
893          </button>
894          <button
895            type="button"
896            :class="{ active: adminTab === 'productos' }"
897            :aria-pressed="adminTab === 'productos'"
898            aria-controls="admin-products"
899            @click="adminTab = 'productos'"
900          >
901            Productos
902          </button>
903        </nav>
904
```

Plantilla: cabecera de pedidos

Líneas 905–935

Carga, mensajes y resumen de cada pedido, cliente, entrega y productos.

Que observar: En la interfaz, el estado del pedido determina que informacion y que acciones se muestran. Vue actualiza esta parte al cambiar los datos; la operacion definitiva se confirma mediante el servicio del backend.

Plantilla: cabecera de pedidos — App.vue · líneas 905–935

```
905         <div v-if= "adminTab === 'pedidos'" id="admin-orders" class="admin-tab-panel">
906           <div class= "admin-variant-heading orders-heading">
907             <div>
908               <h3>Pedidos recibidos</h3>
909               <p>Confirma la disponibilidad para descontar el stock o cancela pedidos que no puedas atender.</p>
910             </div>
911             <button class= "admin-secondary" type="button" :disabled="ordersLoading" @click="loadAdminOrders">
912               {{ ordersLoading ? 'CARGANDO...'  :'Actualizar pedidos'  }}
913             </button>
914           </div>
915           <p v-if= "ordersError" class="admin-message admin-error" role="alert">{{ ordersError }}</p>
916           <p v-if= "ordersSuccess" class="admin-message admin-success" role="status">{{ ordersSuccess }}</p>
917           <p v-if= "ordersLoading && !orders.length" class="admin-message" role="status">Cargando pedidos...</p>
918           <p v-else-if= "!ordersLoading && !ordersError && !orders.length" class="admin-message">TodavÃa no hay pedidos registrados.</p>
919           <div v-if= "orders.length" class="orders-list">
920             <article v-for= "order in orders" :key="order.id_pedido" class="order-card">
921               <header class= "order-card-heading">
922                 <div>
923                   <h4>Pedido #{{ order.id_pedido }}</h4>
924                   <span>{{ formatOrderDate(order.fecha) }}</span>
925                 </div>
926                 <span class= "order-status" :class="`order-status-${order.estado.toLowerCase()}`">{{ order.estado }}</span>
927               </header>
928               <div class= "order-customer">
929                 <strong>{{ order.cliente.nombre }}</strong>
930                 <a :href= "`tel:${order.cliente.telefono}`">{{ order.cliente.telefono }}</a>
931                 <span>{{ order.tipo_entrega === 'retiro'  ?'Retiro en el local'  :`Entrega: ${order.cliente.direccion}`  }}</span>
932               </div>
933               <ul class= "order-lines">
934                 <li v-for= "(item, index) in order.detalles" :key="`${order.id_pedido}-${index}`">
935                   <span>{{ item.nombre }} Â· {{ formatSize(item.categoria, item.talle) }} Â· {{ item.color }} Ã— {{ item.cantidad }}</span>
```

Plantilla: acciones de pedidos

Líneas 936–965

Acciones disponibles según el estado actual del pedido.

Que observar: En la interfaz, el estado del pedido determina que informacion y que acciones se muestran. Vue actualiza esta parte al cambiar los datos; la operacion definitiva se confirma mediante el servicio del backend.

Plantilla: acciones de pedidos — App.vue · líneas 936–965

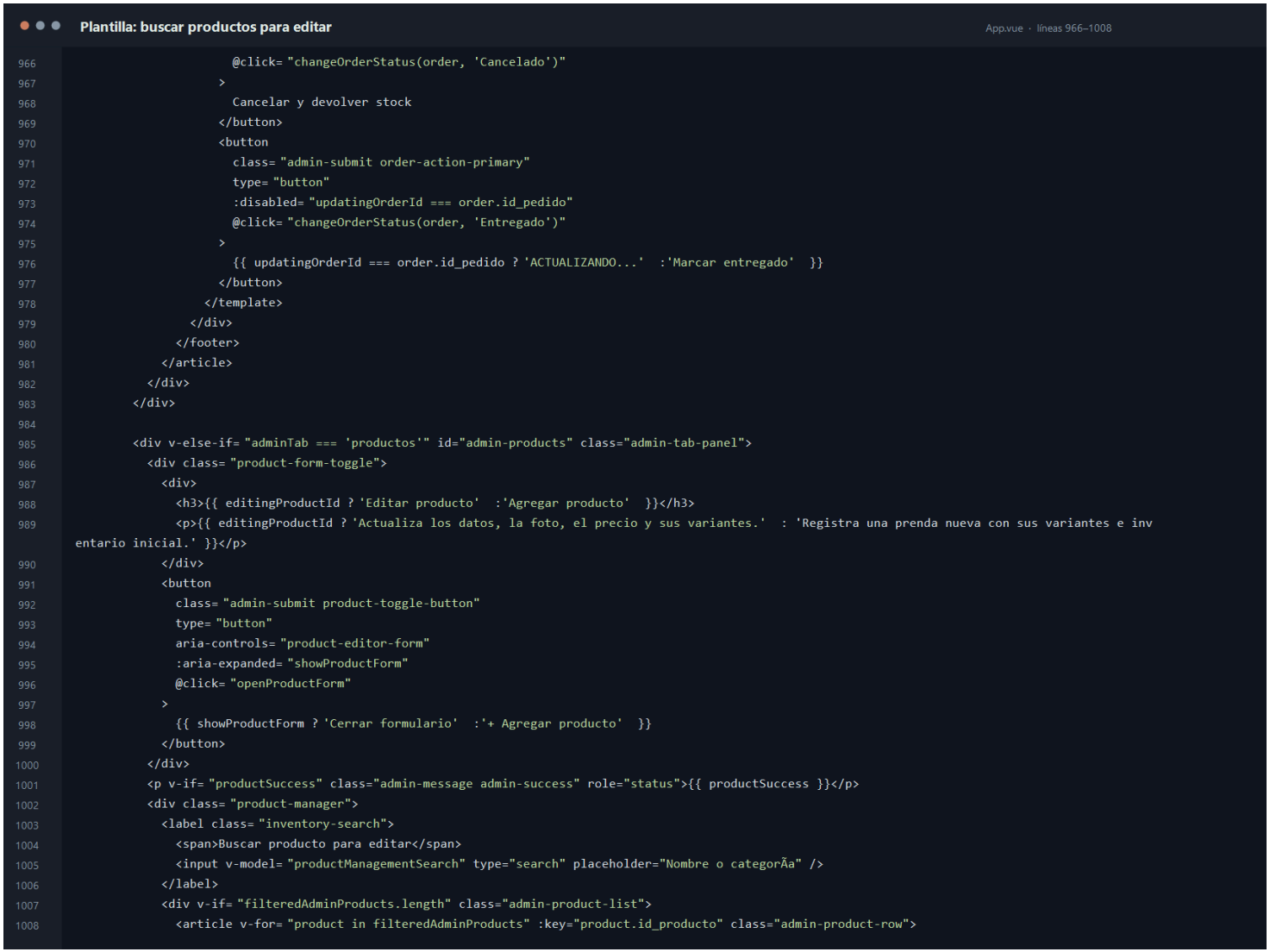
```
936                   <strong>{{ formatPrice(Number(item.precio_unitario) * item.cantidad) }}</strong>
937                 </li>
938               </ul>
939               <p v-if="order.notas" class="order-notes"><strong>Notas:</strong> {{ order.notas }}</p>
940               <footer class="order-card-footer">
941                 <strong>Total {{ formatPrice(order.total) }}</strong>
942                 <div class="order-actions">
943                   <button
944                     v-if="order.estado === 'Pendiente'"
945                     class="admin-secondary"
946                     type="button"
947                     :disabled="updatingOrderId === order.id_pedido"
948                     @click="changeOrderStatus(order, 'Cancelado')"
949                   >
950                     Cancelar
951                   </button>
952                   <button
953                     v-if="order.estado === 'Pendiente'"
954                     class="admin-submit order-action-primary"
955                     type="button"
956                     :disabled="updatingOrderId === order.id_pedido"
957                     @click="changeOrderStatus(order, 'Confirmado')"
958                   >
959                     {{ updatingOrderId === order.id_pedido ? 'ACTUALIZANDO...'  :'Confirmar pedido'  }}
960                   </button>
961                   <template v-else-if="order.estado === 'Confirmado'">
962                     <button
963                       class="admin-secondary"
964                       type="button"
965                       :disabled="updatingOrderId === order.id_pedido"
```

Plantilla: buscar productos para editar

Líneas 966–1008

Búsqueda de productos, miniaturas y botón que abre el editor.

Que observar: Los controles se enlazan con el formulario que se cargo al seleccionar el producto. Asi, la vista previa y los campos muestran el mismo borrador que se enviara al guardar, y cancelar descarta esa edicion.




```
966                     @click="changeOrderStatus(order, 'Cancelado')"
967                   >
968                     Cancelar y devolver stock
969                   </button>
970                   <button
971                     class="admin-submit order-action-primary"
972                     type="button"
973                     :disabled="updatingOrderId === order.id_pedido"
974                     @click="changeOrderStatus(order, 'Entregado')"
975                   >
976                     {{ updatingOrderId === order.id_pedido ? 'ACTUALIZANDO...' :'Marcar entregado' }}
977                   </button>
978                 </template>
979               </div>
980             </footer>
981           </article>
982         </div>
983       </div>
984
985       <div v-else-if="adminTab === 'productos'" id="admin-products" class="admin-tab-panel">
986         <div class="product-form-toggle">
987           <div>
988             <h3>{{ editingProductId ? 'Editar producto' :'Agregar producto' }}</h3>
989             <p>{{ editingProductId ? 'Actualiza los datos, la foto, el precio y sus variantes.' : 'Registra una prenda nueva con sus variantes e inv
    entario inicial.' }}</p>
990           </div>
991           <button
992             class="admin-submit product-toggle-button"
993             type="button"
994             aria-controls="product-editor-form"
995             :aria-expanded="showProductForm"
996             @click="openProductForm"
997           >
998             {{ showProductForm ? 'Cerrar formulario' :'+ Agregar producto' }}
999           </button>
1000        </div>
1001        <p v-if="productSuccess" class="admin-message admin-success" role="status">{{ productSuccess }}</p>
1002        <div class="product-manager">
1003          <label class="inventory-search">
1004            <span>Buscar producto para editar</span>
1005            <input v-model="productManagementSearch" type="search" placeholder="Nombre o categorÃa" />
1006          </label>
1007          <div v-if="filteredAdminProducts.length" class="admin-product-list">
1008            <article v-for="product in filteredAdminProducts" :key="product.id_producto" class="admin-product-row">
```

Plantilla: comienzo del editor

Líneas 1009–1048

Formulario condicional y campos principales, incluida la vista previa de la foto.

Que observar: Los controles se enlazan con el formulario que se cargo al seleccionar el producto. Asi, la vista previa y los campos muestran el mismo borrador que se enviara al guardar, y cancelar descarta esa edicion.

Plantilla: comienzo del editor — App.vue · líneas 1009–1048

```
1009              <img
1010                :src="adminProductImage(product)"
1011                :alt="`Imagen de ${product.nombre}`"
1012                class="admin-product-image"
1013                @error="handleImageError"
1014              />
1015              <div class="admin-product-copy">
1016                <strong>{{ product.nombre }}</strong>
1017                <span>{{ product.categoria }} Â· {{ formatPrice(product.precio) }} Â· {{ product.variantes?.length || 0 }} variantes</span>
1018              </div>
1019              <button class="admin-secondary admin-product-edit" type="button" @click="editProduct(product)">
1020                Editar
1021              </button>
1022            </article>
1023          </div>
1024          <p v-else class="inventory-help">No encontramos productos con esa bÃºsqueda.</p>
1025        </div>
1026        <form
1027          v-if="showProductForm"
1028          id="product-editor-form"
1029          ref="productEditorForm"
1030          class="admin-form product-create-form"
1031          @submit.prevent="submitProduct"
1032        >
1033          <div class="admin-fields">
1034            <label class="admin-field">
1035              <span>Nombre *</span>
1036              <input ref="productNameInput" v-model.trim="productForm.nombre" type="text" maxlength="255" required />
1037            </label>
1038            <label class="admin-field">
1039              <span>CategorÃa *</span>
1040              <input v-model.trim="productForm.categoria" type="text" list="admin-categories" maxlength="255" required />
1041              <datalist id="admin-categories">
1042                <option value="Mujer" />
1043                <option value="Hombre" />
1044                <option value="Unisex" />
1045              </datalist>
1046            </label>
1047            <label class="admin-field">
1048              <span>Precio *</span>
```

Plantilla: variantes del producto

Líneas 1049–1110

Edición de variantes y botones para guardar cambios o cancelar.

Que observar: Los controles se enlazan con el formulario que se cargo al seleccionar el producto. Asi, la vista previa y los campos muestran el mismo borrador que se enviara al guardar, y cancelar descarta esa edicion.




```
                <input v-model="productForm.precio" type="number" min="0.01" max="99999999.99" step="0.01" required />
              </label>
              <label class="admin-field">
                <span>Imagen del producto</span>
                <input
                  ref="productImageInput"
                  class="image-file-input"
                  type="file"
                  accept=".jpg,.jpeg,.png,.webp,image/jpeg,image/png,image/webp"
                  @change="handleProductImageSelection"
                />
                <span class="image-upload-hint">JPG, PNG o WebP Â· mÃ¡ximo 5 MB</span>
                <div v-if="productImagePreview || productForm.imagen" class="image-preview-row">
                  <img :src="productImagePreview || productForm.imagen" alt="Vista previa de la imagen del producto" class="image-preview" @error="handleImageError" />
                  <button class="admin-secondary" type="button" @click="removeProductImage">Quitar imagen</button>
                </div>
                <input v-model.trim="productForm.imagen" type="url" maxlength="255" placeholder="O pega una URL de imagen" />
              </label>
              <label class="admin-field admin-field-wide">
                <span>DescripciÃ³n</span>
                <textarea v-model.trim="productForm.descripcion" rows="3"></textarea>
              </label>
            </div>

            <div class="admin-variant-heading">
              <div>
                <h3>Variantes *</h3>
                <p>Al menos una; mÃ¡ximo 40. Las variantes incluidas en pedidos anteriores no se pueden quitar ni cambiar de talle o color.</p>
              </div>
              <button class="admin-secondary" type="button" @click="addVariant">AÃ±adir variante</button>
            </div>

            <div class="admin-variants">
              <div v-for="(variant, index) in productForm.variantes" :key="index" class="admin-variant-row">
                <label class="admin-field">
                  <span>Talle *</span>
                  <input v-model.trim="variant.talle" type="text" maxlength="255" placeholder="M" required />
                </label>
                <label class="admin-field">
                  <span>Color *</span>
                  <input v-model.trim="variant.color" type="text" maxlength="255" placeholder="Negro" required />
                </label>
                <label class="admin-field">
                  <span>Stock *</span>
                  <input v-model.number="variant.stock" type="number" min="0" max="2147483647" step="1" required />
                </label>
                <button class="admin-remove-variant" type="button" :disabled="productForm.variantes.length === 1" aria-label="Eliminar variante" @click="removeVariant(index)">Ã—</button>
              </div>
            </div>

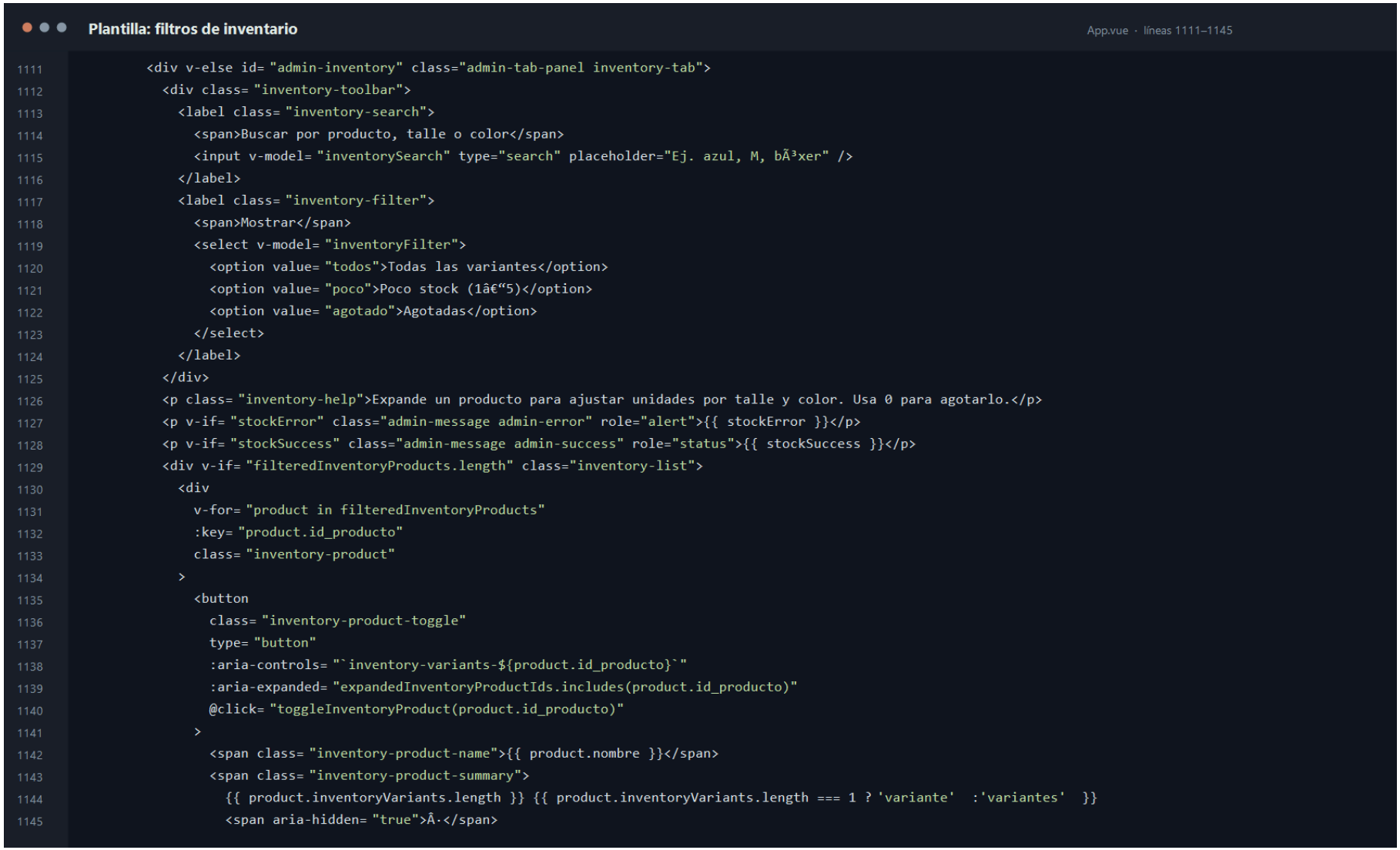
            <p v-if="productError" class="admin-message admin-error" role="alert">{{ productError }}</p>
            <div class="product-form-actions">
              <button class="admin-submit" type="submit" :disabled="productLoading">
                {{ productLoading ? 'GUARDANDO...' : editingProductId ?'GUARDAR CAMBIOS' :'CREAR PRODUCTO' }}
              </button>
              <button class="admin-secondary product-cancel-button" type="button" :disabled="productLoading" @click="closeProductForm">
                Cancelar
              </button>
            </div>
          </form>
        </div>
```

Plantilla: filtros de inventario

Líneas 1111–1145

Búsqueda, filtros, mensajes de estado y expansión de un producto.

Que observar: Los valores que se editan corresponden a variantes concretas, no solo al producto general. Los controles reflejan el estado del panel y conectan la accion de guardar con la funcion que actualiza el servidor.



Plantilla: ajustar stock

Líneas 1146–1198

Entrada de unidades, indicador de disponibilidad y botón de guardado por variante.

Que observar: Los valores que se editan corresponden a variantes concretas, no solo al producto general. Los controles reflejan el estado del panel y conectan la accion de guardar con la funcion que actualiza el servidor.




```
1146                  {{ productStockTotal(product, product.inventoryVariants) }} unidades
1147                </span>
1148                <span class="inventory-expand-mark" aria-hidden="true">{{ expandedInventoryProductIds.includes(product.id_producto) ?'â¯'  :'+'  }}</
    span>
1149              </button>
1150              <div
1151                v-if="expandedInventoryProductIds.includes(product.id_producto)"
1152                :id="`inventory-variants-${product.id_producto}`"
1153                class="inventory-variants"
1154              >
1155                <div
1156                  v-for="variant in product.inventoryVariants"
1157                  :key="variant.id_variante"
1158                  class="inventory-row"
1159                >
1160                  <span class="inventory-variant">{{ formatSize(product.categoria, variant.talle) }} Â· {{ variant.color }}</span>
1161                  <label class="inventory-stock-field" :for="`stock-${variant.id_variante}`">
1162                    <span>Unidades</span>
1163                    <input
1164                      :id="`stock-${variant.id_variante}`"
1165                      v-model.number="stockDrafts[variant.id_variante]"
1166                      type="number"
1167                      min="0"
1168                      max="2147483647"
1169                      step="1"
1170                      required
1171                      :disabled="stockLoading"
1172                    />
1173                  </label>
1174                  <span
1175                    class="inventory-status"
1176                    :class="{
1177    'inventory-out-of-stock'                    : Number(stockDrafts[variant.id_variante]) === 0,
1178    'inventory-low-stock'                     : Number(stockDrafts[variant.id_variante]) > 0 && Number(stockDrafts[variant.id_variante]) <= 5,
1179                    }"
1180                  >
1181                    {{ Number(stockDrafts[variant.id_variante]) === 0 ? 'Agotado'  : Number(stockDrafts[variant.id_variante]) <= 5 ?'Poco stock'  : 'Di
    sponible' }}
1182                  </span>
1183                  <button
1184                    class="admin-secondary inventory-save"
1185                    type="button"
1186                    :disabled="stockLoading || Number(stockDrafts[variant.id_variante]) === Number(variant.stock)"
1187                    @click="saveVariantStock(variant)"
1188                  >
1189                    {{ stockLoading ? 'GUARDANDO...'  :'Guardar'  }}
1190                  </button>
1191                </div>
1192              </div>
1193            </div>
1194          </div>
1195          <p v-else-if="!products.length" class="admin-message">TodavÃa no hay productos para administrar.</p>
1196          <p v-else class="admin-message">No encontramos variantes con esos filtros.</p>
1197        </div>
1198      </section>
```

Plantilla: portada

Líneas 1200–1218

Primera sección pública y su imagen principal.

Que observar: La plantilla decide que mostrar segun el estado actual y conecta botones o campos con las acciones del componente. Esto separa lo que la persona ve de las operaciones que actualizan los datos.

Plantilla: portada — App.vue · líneas 1200–1218

```
1200      <section id="inicio" class="hero-section">
1201        <div class="hero-copy">
1202          <p class="eyebrow"><span class="eyebrow-line"></span> PARA TODOS TUS DÃAS</p>
1203          <h1>Lo que llevas<br />tambiÃ©n <span>se siente.</span></h1>
1204          <p class="hero-description">Prendas hechas para acompaÃ±arte con comodidad, confianza y un poquito mÃ¡s de energÃa.</p>
1205          <a class="hero-link" href="#catalogo">Descubre la coleccIÃ³n <span aria-hidden="true">â†˜</span></a>
1206          <div class="hero-index"><span>01</span><span class="index-rule"></span><span>BUENA ENERGÃA</span></div>
1207        </div>
1208        <div class="hero-image-wrap">
1209          <img
1210            class="hero-image"
1211            :src="products[6]?.imagen_url || fallbackImage"
1212            alt="Prendas de la coleccIÃ³n Buena EnergÃa"
1213            @error="handleImageError"
1214          />
1215          <div class="hero-stamp" aria-label="Hecho para sentirte bien">
1216            <span>HECHO PARA</span>
1217            <strong>SENTIRTE<br />BIEN</strong>
1218            <span aria-hidden="true">âœ³</span>
```

Plantilla: búsqueda y categorías

Líneas 1219–1237

Campo de búsqueda, filtros de categoría y contador de resultados.

Que observar: Esta parte publica reacciona al catalogo y a los filtros elegidos. Las directivas de Vue enlazan los controles y acciones con el estado, por lo que la pagina refleja la seleccion sin recargar el sitio.

Plantilla: bÃºsqueda y categorÃas — App.vue · líneas 1219–1237

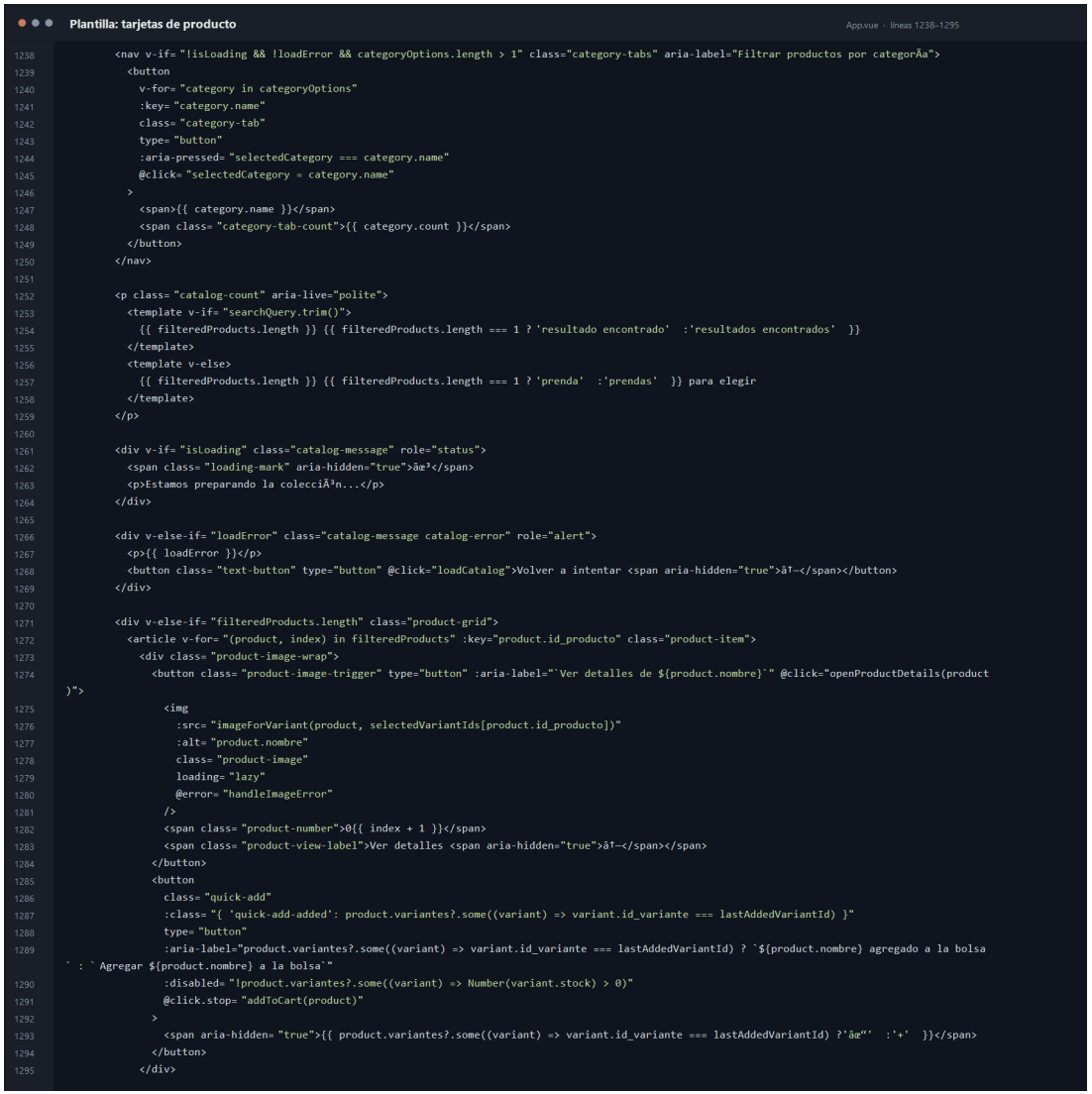
```
1219            </div>
1220            <span class= "image-caption">COMODIDAD QUE VA CONTIGO</span>
1221          </div>
1222          <div class= "hero-side-note">BUENA ENERGÃ▯A Â· DESDE ADENTRO</div>
1223        </section>
1224
1225        <section id= "catalogo" class="catalog-section">
1226          <div class= "section-heading">
1227            <div>
1228              <p class= "eyebrow"><span class="eyebrow-line"></span> NUESTRA SELECCIÃ“N</p>
1229              <h2>Un buen comienzo.</h2>
1230            </div>
1231            <label class= "search-field">
1232              <span class= "search-icon" aria-hidden="true">â€•</span>
1233              <input v-model= "searchQuery" type="search" placeholder="Nombre, categorÃa, talle o color" aria-label="Buscar por nombre, categorÃa, tall
e o color" />
1234              <button v-if= "searchQuery" class="clear-search" type="button" aria-label="Limpiar bÃºsqueda" @click="searchQuery = ''">Ã—</button>
1235            </label>
1236          </div>
1237
```

Plantilla: tarjetas de producto

Líneas 1238–1295

Imagen de variante, detalle, elección de talle y botón de agregado rápido.

Que observar: Esta parte publica reacciona al catalogo y a los filtros elegidos. Las directivas de Vue enlazan los controles y acciones con el estado, por lo que la pagina refleja la seleccion sin recargar el sitio.




```
1238        <nav v-if="!isLoading && !loadError && categoryOptions.length > 1" class="category-tabs" aria-label="Filtrar productos por categorÃa">
1239          <button
1240            v-for="category in categoryOptions"
1241            :key="category.name"
1242            class="category-tab"
1243            type="button"
1244            :aria-pressed="selectedCategory === category.name"
1245            @click="selectedCategory = category.name"
1246          >
1247            <span>{{ category.name }}</span>
1248            <span class="category-tab-count">{{ category.count }}</span>
1249          </button>
1250        </nav>
1251
1252        <p class="catalog-count" aria-live="polite">
1253          <template v-if="searchQuery.trim()">
1254            {{ filteredProducts.length }} {{ filteredProducts.length === 1 ? 'resultado encontrado' : 'resultados encontrados' }}
1255          </template>
1256          <template v-else>
1257            {{ filteredProducts.length }} {{ filteredProducts.length === 1 ? 'prenda' : 'prendas' }} para elegir
1258          </template>
1259        </p>
1260
1261        <div v-if="isLoading" class="catalog-message" role="status">
1262          <span class="loading-mark" aria-hidden="true">â¾³</span>
1263          <p>Estamos preparando la colecciÃ³n...</p>
1264        </div>
1265
1266        <div v-else-if="loadError" class="catalog-message catalog-error" role="alert">
1267          <p>{{ loadError }}</p>
1268          <button class="text-button" type="button" @click="loadCatalog">Volver a intentar <span aria-hidden="true">â†—</span></button>
1269        </div>
1270
1271        <div v-else-if="filteredProducts.length" class="product-grid">
1272          <article v-for="(product, index) in filteredProducts" :key="product.id_producto" class="product-item">
1273            <div class="product-image-wrap">
1274              <button class="product-image-trigger" type="button" :aria-label="`Ver detalles de ${product.nombre}`" @click="openProductDetails(product
     )">
1275                <img
1276                  :src="imageForVariant(product, selectedVariantIds[product.id_producto])"
1277                  :alt="product.nombre"
1278                  class="product-image"
1279                  loading="lazy"
1280                  @error="handleImageError"
1281                />
1282                <span class="product-number">0{{ index + 1 }}</span>
1283                <span class="product-view-label">Ver detalles <span aria-hidden="true">â†—</span></span>
1284              </button>
1285              <button
1286                class="quick-add"
1287                :class="{ 'quick-add-added': product.variantes?.some((variant) => variant.id_variante === lastAddedVariantId) }"
1288                type="button"
1289                :aria-label="product.variantes?.some((variant) => variant.id_variante === lastAddedVariantId) ? `${product.nombre} agregado a la bolsa
     ` : `Agregar ${product.nombre} a la bolsa`"
1290                :disabled="!product.variantes?.some((variant) => Number(variant.stock) > 0)"
1291                @click.stop="addToCart(product)"
1292              >
1293                <span aria-hidden="true">{{ product.variantes?.some((variant) => variant.id_variante === lastAddedVariantId) ?'â œ“' :'+' }}</span>
1294              </button>
1295            </div>
```

Plantilla: estado sin resultados

Líneas 1296–1320

Mensajes cuando no hay coincidencias y acción para volver a la colección.

Que observar: Esta parte publica reacciona al catalogo y a los filtros elegidos. Las directivas de Vue enlazan los controles y acciones con el estado, por lo que la pagina refleja la seleccion sin recargar el sitio.




```
1296            <label v-if= "product.variantes?.length" class="variant-picker">
1297              <span>Talle y color</span>
1298              <select v-model.number= "selectedVariantIds[product.id_producto]" :aria-label="`Variante de ${product.nombre}`">
1299                <option v-for= "variant in product.variantes" :key="variant.id_variante" :value="variant.id_variante" :disabled="Number(variant.stock)
< 1">
1300                  {{ formatSize(product.categoria, variant.talle) }} Â· {{ variant.color }}{{ Number(variant.stock) < 1 ? ' Â· Agotado'  : ` Â· ${varia
nt.stock} disponibles` }}
1301                </option>
1302              </select>
1303            </label>
1304            <div class= "product-details">
1305              <div>
1306                <h3><button class= "product-title-trigger" type="button" @click="openProductDetails(product)">{{ product.nombre }}</button></h3>
1307                <p>{{ product.descripcion }}</p>
1308              </div>
1309              <span class= "product-price">{{ formatPrice(product.precio) }}</span>
1310            </div>
1311            <div v-if= "product.colors?.length" class="color-list" aria-label="Colores disponibles">
1312              <span
1313                v-for= "color in product.colors"
1314                :key= "color.name"
1315                class= "color-swatch"
1316                :style= "{ '--swatch-color': color.hex || '#d6d6d2' }"
1317                :title= "color.name"
1318              ></span>
1319              <span class= "color-caption">{{ product.colors.length }} {{ product.colors.length === 1 ?'color'  :'colores'  }}</span>
1320            </div>
```

Plantilla: marca y contacto

Líneas 1321–1358

Secciones de identidad, contacto y pie de página.

Que observar: La plantilla decide que mostrar segun el estado actual y conecta botones o campos con las acciones del componente. Esto separa lo que la persona ve de las operaciones que actualizan los datos.

Plantilla: marca y contacto — App.vue · líneas 1321–1358

```
1321          </article>
1322        </div>
1323
1324        <div v-else class="catalog-message">
1325          <p v-if="searchQuery">No encontramos prendas con â€œ{{ searchQuery }}â€.</p>
1326          <p v-else>No hay prendas en la categorÃa â€œ{{ selectedCategory }}â€.</p>
1327          <button class="text-button" type="button" @click="searchQuery = ''; selectedCategory = 'Todos'">Ver toda la colecciÃ³n <span aria-hidden="tr
ue">â†—</span></button>
1328        </div>
1329      </section>
1330
1331      <section id="historia" class="brand-note">
1332        <span class="brand-note-mark" aria-hidden="true">âœ³</span>
1333        <p>Sentirte bien empieza<br />con lo que eliges <em>para ti.</em></p>
1334        <span class="brand-note-signature">BUENA ENERGÃA, SIEMPRE.</span>
1335      </section>
1336
1337      <section id="contacto" class="contact-section" aria-labelledby="contact-title">
1338        <div class="contact-copy">
1339          <p class="eyebrow"><span class="eyebrow-line"></span> CONTACTO DIRECTO</p>
1340          <h2 id="contact-title">Â¿Te ayudamos con tu pedido?</h2>
1341          <p>ComunÃcate con Fernanda para consultas sobre talles, prendas o disponibilidad.</p>
1342          <a class="contact-phone" href="tel:094449306">Llamar al 094 449 306 <span aria-hidden="true">â†—</span></a>
1343        </div>
1344        <img
1345          class="contact-image"
1346          src="/contacto-buena-energia.jpeg"
1347          alt="Tarjeta de contacto de Buena EnergÃa con Fernanda y el telÃ©fono 094 449 306"
1348          loading="lazy"
1349        />
1350      </section>
1351    </main>
1352
1353    <footer class="site-footer">
1354      <a class="footer-wordmark" href="#inicio">BUENA ENERGÃA<span>.</span></a>
1355      <span>Prendas para vivir a tu manera.</span>
1356      <a href="#catalogo">Volver arriba â†‘</a>
1357    </footer>
1358
```

Plantilla: detalle del producto

Líneas 1359–1396

Ventana con imagen, precio, stock y acción de agregar al carrito.

Que observar: Los campos y botones de esta ventana trabajan sobre el producto o la compra seleccionados. Las condiciones de Vue controlan cuando aparece cada elemento y las acciones delegan la validacion al flujo correspondiente.

Plantilla: detalle del producto — App.vue · líneas 1359–1396

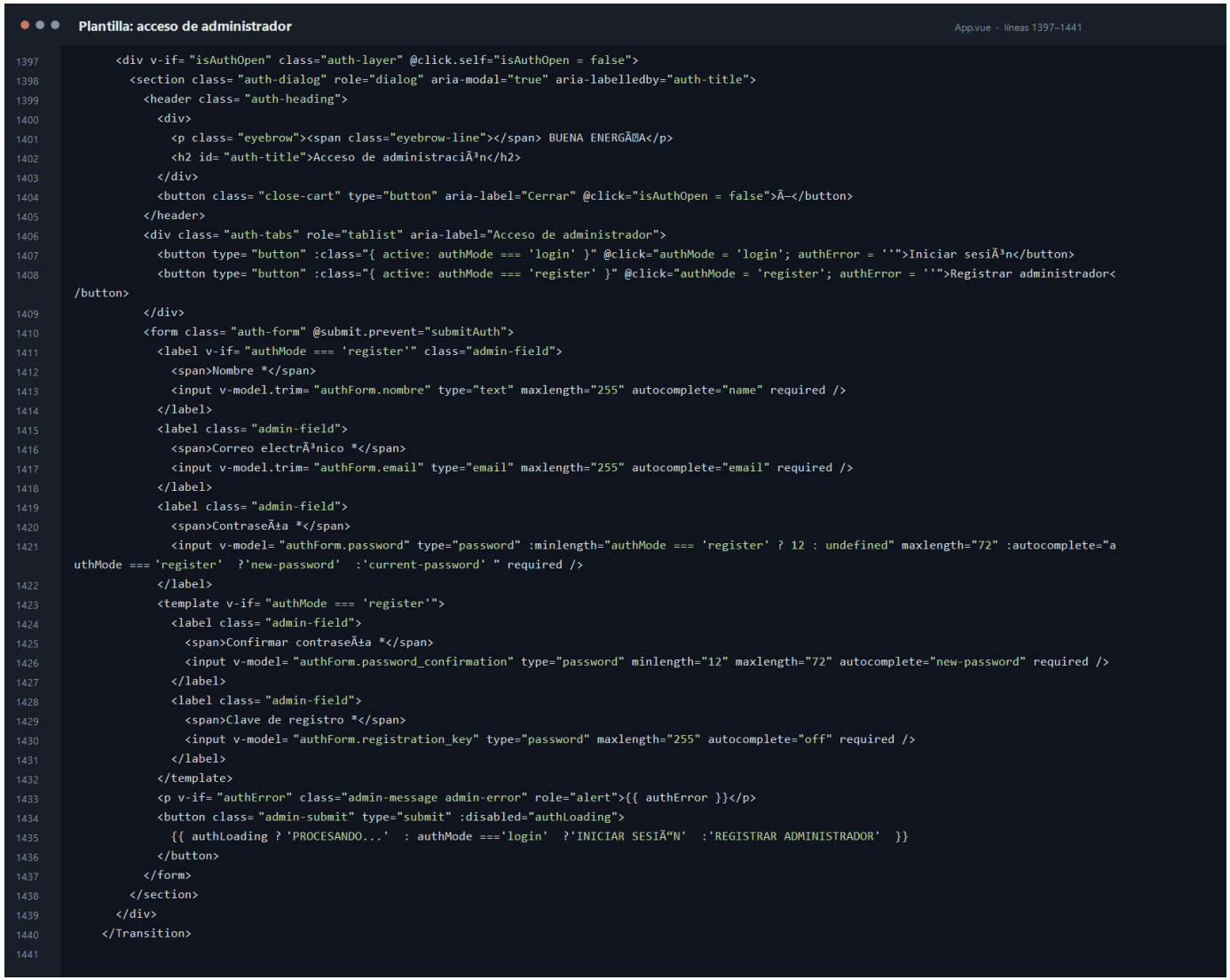
```
1359      <Transition name="auth-modal">
1360        <div v-if="selectedProduct" class="auth-layer product-detail-layer" @click.self="selectedProduct = null">
1361          <section class="product-detail-dialog" role="dialog" aria-modal="true" aria-labelledby="detail-title">
1362            <button class="close-cart detail-close" type="button" aria-label="Cerrar detalles" @click="selectedProduct = null">Ã—</button>
1363            <div class="product-detail-layout">
1364              <div class="product-detail-image-wrap">
1365                <img
1366                  :src="detailImage"
1367                  :alt="selectedProduct.nombre"
1368                  @error="handleImageError"
1369                />
1370              </div>
1371              <div class="product-detail-copy">
1372                <p class="eyebrow"><span class="eyebrow-line"></span> {{ selectedProduct.categoria ||'BUENA ENERGÃ‹A'  }}</p>
1373                <h2 id="detail-title">{{ selectedProduct.nombre }}</h2>
1374                <p class="product-detail-description">{{ selectedProduct.descripcion ||'Una prenda pensada para acompaÃ±arte todos los dÃas.'  }}</p>
1375                <strong class="product-detail-price">{{ formatPrice(selectedProduct.precio) }}</strong>
1376                <label class="detail-variant-picker">
1377                  <span>Talle y color</span>
1378                  <select v-model.number="detailVariantId">
1379                    <option v-for="variant in selectedProduct.variantes || []" :key="variant.id_variante" :value="variant.id_variante" :disabled="Number
(variant.stock) < 1">
1380                      {{ formatSize(selectedProduct.categoria, variant.talle) }} Â· {{ variant.color }}{{ Number(variant.stock) < 1 ? ' Â· Agotado'  : `
Â· ${variant.stock} disponibles` }}
1381                    </option>
1382                  </select>
1383                </label>
1384                <p v-if="detailVariant && Number(detailVariant.stock) > 0" class="detail-stock">Disponible para agregar al carrito</p>
1385                <p v-else class="detail-stock detail-out-of-stock">Esta prenda estÃ¡ agotada</p>
1386                <p v-if="cartNotice" class="cart-notice" :class="`cart-notice-${cartNoticeType}`" role="status">{{ cartNotice }}</p>
1387                <button class="detail-add-button" type="button" :disabled="!detailVariant || Number(detailVariant.stock) < 1" @click="addDetailToCart">
1388                  Agregar al carrito
1389                </button>
1390              </div>
1391            </div>
1392          </section>
1393        </div>
1394      </Transition>
1395
1396      <Transition name="auth-modal">
```

Plantilla: acceso de administrador

Líneas 1397–1441

Formulario de acceso o registro, con los campos de registro condicionales.

Que observar: La plantilla decide que mostrar segun el estado actual y conecta botones o campos con las acciones del componente. Esto separa lo que la persona ve de las operaciones que actualizan los datos.



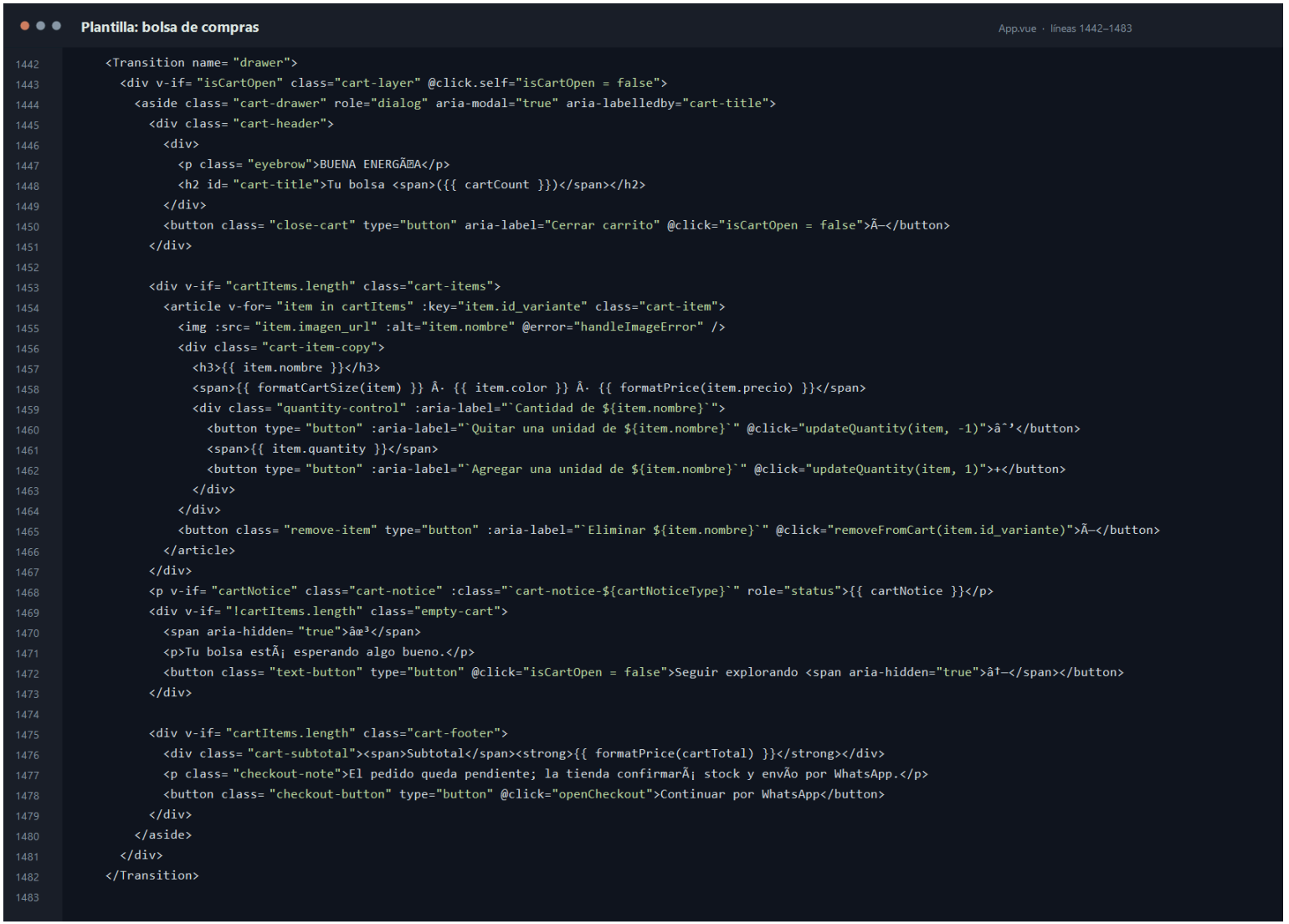

```
1397      <div v-if="isAuthOpen" class="auth-layer" @click.self="isAuthOpen = false">
1398        <section class="auth-dialog" role="dialog" aria-modal="true" aria-labelledby="auth-title">
1399          <header class="auth-heading">
1400            <div>
1401              <p class="eyebrow"><span class="eyebrow-line"></span> BUENA ENERGÃ©A</p>
1402              <h2 id="auth-title">Acceso de administraciÃ³n</h2>
1403            </div>
1404            <button class="close-cart" type="button" aria-label="Cerrar" @click="isAuthOpen = false">Ã—</button>
1405          </header>
1406          <div class="auth-tabs" role="tablist" aria-label="Acceso de administrador">
1407            <button type="button" :class="{ active: authMode === 'login' }" @click="authMode = 'login'; authError = ''">Iniciar sesiÃ³n</button>
1408            <button type="button" :class="{ active: authMode === 'register' }" @click="authMode = 'register'; authError = ''">Registrar administrador<
/button>
1409          </div>
1410          <form class="auth-form" @submit.prevent="submitAuth">
1411            <label v-if="authMode === 'register'" class="admin-field">
1412              <span>Nombre *</span>
1413              <input v-model.trim="authForm.nombre" type="text" maxlength="255" autocomplete="name" required />
1414            </label>
1415            <label class="admin-field">
1416              <span>Correo electrÃ³nico *</span>
1417              <input v-model.trim="authForm.email" type="email" maxlength="255" autocomplete="email" required />
1418            </label>
1419            <label class="admin-field">
1420              <span>ContraseÃ±a *</span>
1421              <input v-model="authForm.password" type="password" :minlength="authMode === 'register' ? 12 : undefined" maxlength="72" :autocomplete="a
uthMode === 'register'  ?'new-password'  :'current-password' " required />
1422            </label>
1423            <template v-if="authMode === 'register'">
1424              <label class="admin-field">
1425                <span>Confirmar contraseÃ±a *</span>
1426                <input v-model="authForm.password_confirmation" type="password" minlength="12" maxlength="72" autocomplete="new-password" required />
1427              </label>
1428              <label class="admin-field">
1429                <span>Clave de registro *</span>
1430                <input v-model="authForm.registration_key" type="password" maxlength="255" autocomplete="off" required />
1431              </label>
1432            </template>
1433            <p v-if="authError" class="admin-message admin-error" role="alert">{{ authError }}</p>
1434            <button class="admin-submit" type="submit" :disabled="authLoading">
1435              {{ authLoading ? 'PROCESANDO...'  : authMode ==='login'  ?'INICIAR SESIÃ“N'  :'REGISTRAR ADMINISTRADOR'  }}
1436            </button>
1437          </form>
1438        </section>
1439      </div>
1440    </Transition>
1441
```

Plantilla: bolsa de compras

Líneas 1442–1483

Artículos, cantidades, subtotal y acceso al checkout.

Que observar: Los campos y botones de esta ventana trabajan sobre el producto o la compra seleccionados. Las condiciones de Vue controlan cuando aparece cada elemento y las acciones delegan la validacion al flujo correspondiente.



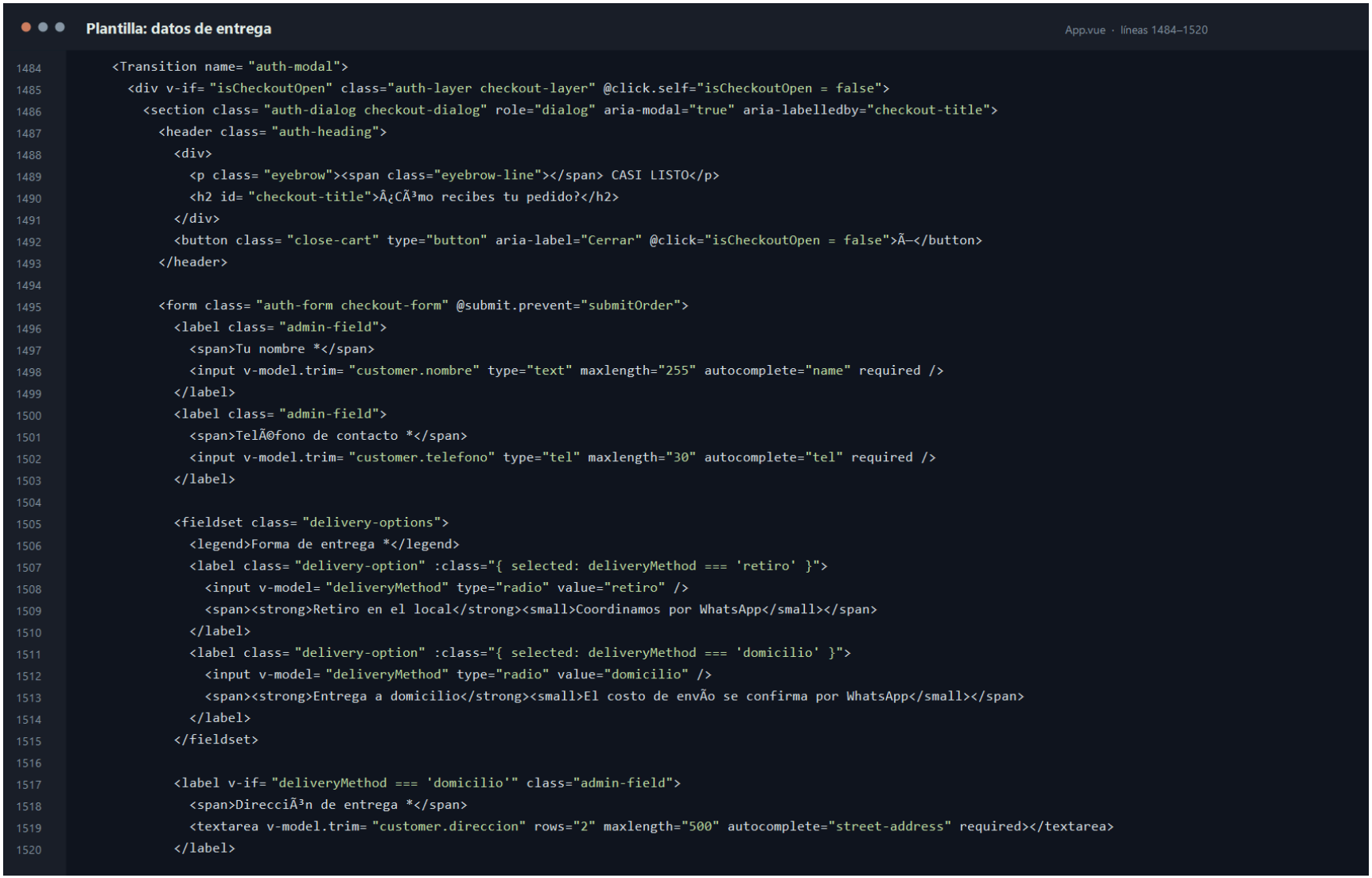

```
1442    <Transition name="drawer">
1443      <div v-if="isCartOpen" class="cart-layer" @click.self="isCartOpen = false">
1444        <aside class="cart-drawer" role="dialog" aria-modal="true" aria-labelledby="cart-title">
1445          <div class="cart-header">
1446            <div>
1447              <p class="eyebrow">BUENA ENERGÃ⊞A</p>
1448              <h2 id="cart-title">Tu bolsa <span>({{ cartCount }})</span></h2>
1449            </div>
1450            <button class="close-cart" type="button" aria-label="Cerrar carrito" @click="isCartOpen = false">Ã—</button>
1451          </div>
1452
1453          <div v-if="cartItems.length" class="cart-items">
1454            <article v-for="item in cartItems" :key="item.id_variante" class="cart-item">
1455              <img :src="item.imagen_url" :alt="item.nombre" @error="handleImageError" />
1456              <div class="cart-item-copy">
1457                <h3>{{ item.nombre }}</h3>
1458                <span>{{ formatCartSize(item) }} Â· {{ item.color }} Â· {{ formatPrice(item.precio) }}</span>
1459                <div class="quantity-control" :aria-label="`Cantidad de ${item.nombre}`">
1460                  <button type="button" :aria-label="`Quitar una unidad de ${item.nombre}`" @click="updateQuantity(item, -1)">â^'</button>
1461                  <span>{{ item.quantity }}</span>
1462                  <button type="button" :aria-label="`Agregar una unidad de ${item.nombre}`" @click="updateQuantity(item, 1)">+</button>
1463                </div>
1464              </div>
1465              <button class="remove-item" type="button" :aria-label="`Eliminar ${item.nombre}`" @click="removeFromCart(item.id_variante)">Ã—</button>
1466            </article>
1467          </div>
1468          <p v-if="cartNotice" class="cart-notice" :class="`cart-notice-${cartNoticeType}`" role="status">{{ cartNotice }}</p>
1469          <div v-if="!cartItems.length" class="empty-cart">
1470            <span aria-hidden="true">â€³</span>
1471            <p>Tu bolsa estÃ¡ esperando algo bueno.</p>
1472            <button class="text-button" type="button" @click="isCartOpen = false">Seguir explorando <span aria-hidden="true">â†—</span></button>
1473          </div>
1474
1475          <div v-if="cartItems.length" class="cart-footer">
1476            <div class="cart-subtotal"><span>Subtotal</span><strong>{{ formatPrice(cartTotal) }}</strong></div>
1477            <p class="checkout-note">El pedido queda pendiente; la tienda confirmarÃ¡ stock y envÃo por WhatsApp.</p>
1478            <button class="checkout-button" type="button" @click="openCheckout">Continuar por WhatsApp</button>
1479          </div>
1480        </aside>
1481      </div>
1482    </Transition>
1483
```

Plantilla: datos de entrega

Líneas 1484–1520

Nombre, teléfono, método de entrega, dirección y notas del pedido.

Que observar: Los campos y botones de esta ventana trabajan sobre el producto o la compra seleccionados. Las condiciones de Vue controlan cuando aparece cada elemento y las acciones delegan la validacion al flujo correspondiente.



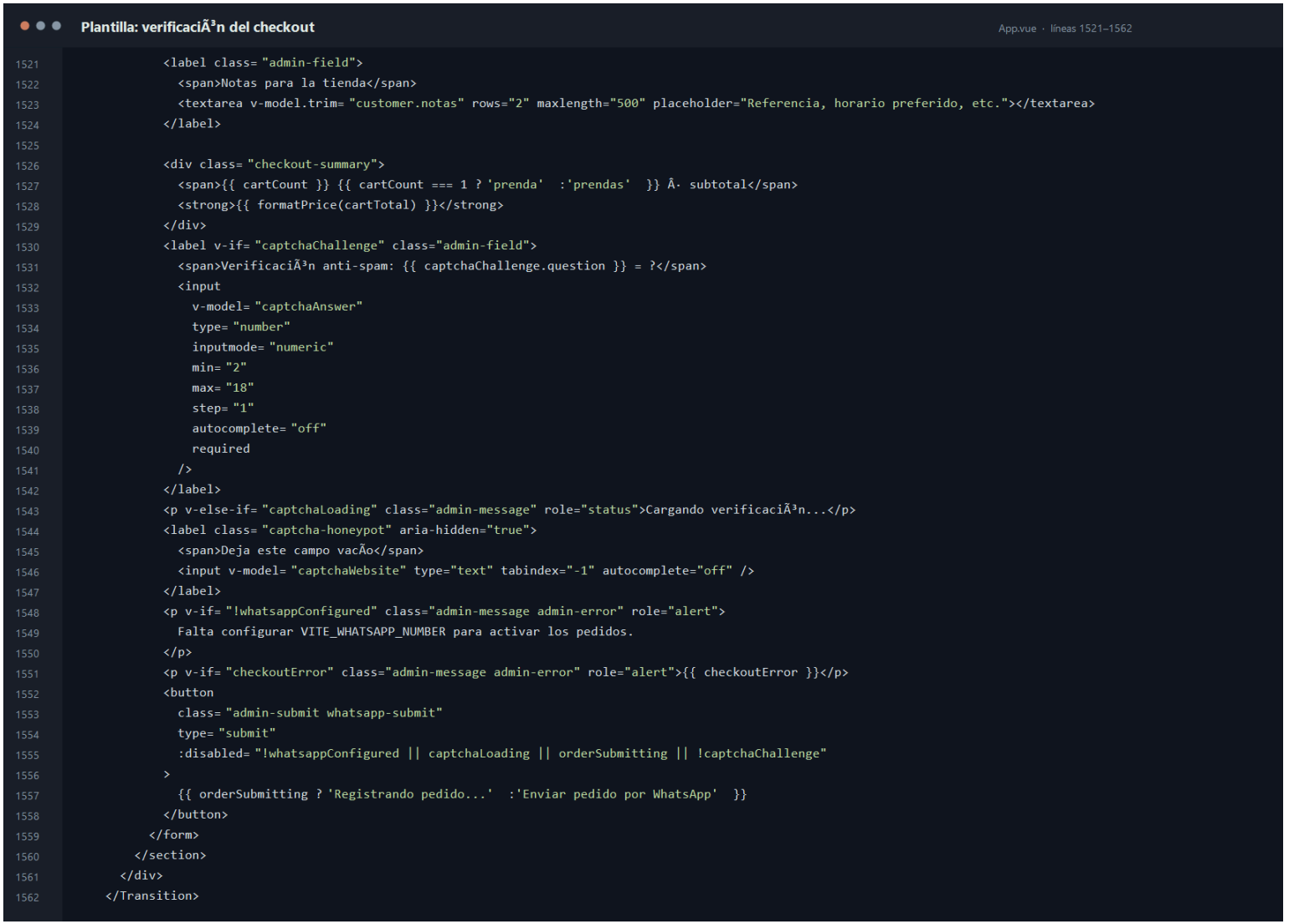

```
1484      <Transition name="auth-modal">
1485        <div v-if="isCheckoutOpen" class="auth-layer checkout-layer" @click.self="isCheckoutOpen = false">
1486          <section class="auth-dialog checkout-dialog" role="dialog" aria-modal="true" aria-labelledby="checkout-title">
1487            <header class="auth-heading">
1488              <div>
1489                <p class="eyebrow"><span class="eyebrow-line"></span> CASI LISTO</p>
1490                <h2 id="checkout-title">Â¿CÃ³mo recibes tu pedido?</h2>
1491              </div>
1492              <button class="close-cart" type="button" aria-label="Cerrar" @click="isCheckoutOpen = false">Ã—</button>
1493            </header>
1494
1495            <form class="auth-form checkout-form" @submit.prevent="submitOrder">
1496              <label class="admin-field">
1497                <span>Tu nombre *</span>
1498                <input v-model.trim="customer.nombre" type="text" maxlength="255" autocomplete="name" required />
1499              </label>
1500              <label class="admin-field">
1501                <span>TelÃ©fono de contacto *</span>
1502                <input v-model.trim="customer.telefono" type="tel" maxlength="30" autocomplete="tel" required />
1503              </label>
1504
1505              <fieldset class="delivery-options">
1506                <legend>Forma de entrega *</legend>
1507                <label class="delivery-option" :class="{ selected: deliveryMethod === 'retiro' }">
1508                  <input v-model="deliveryMethod" type="radio" value="retiro" />
1509                  <span><strong>Retiro en el local</strong><small>Coordinamos por WhatsApp</small></span>
1510                </label>
1511                <label class="delivery-option" :class="{ selected: deliveryMethod === 'domicilio' }">
1512                  <input v-model="deliveryMethod" type="radio" value="domicilio" />
1513                  <span><strong>Entrega a domicilio</strong><small>El costo de envÃo se confirma por WhatsApp</small></span>
1514                </label>
1515              </fieldset>
1516
1517              <label v-if="deliveryMethod === 'domicilio'" class="admin-field">
1518                <span>DirecciÃ³n de entrega *</span>
1519                <textarea v-model.trim="customer.direccion" rows="2" maxlength="500" autocomplete="street-address" required></textarea>
1520              </label>
```

Plantilla: verificación del checkout

Líneas 1521–1562

Resumen, CAPTCHA, honeypot, errores y botón para enviar el pedido.

Que observar: Los campos y botones de esta ventana trabajan sobre el producto o la compra seleccionados. Las condiciones de Vue controlan cuando aparece cada elemento y las acciones delegan la validacion al flujo correspondiente.



Plantilla: feedback y acceso rápido

Líneas 1563–1585

Aviso temporal al agregar y botón flotante para volver a la bolsa.

Que observar: La plantilla decide que mostrar segun el estado actual y conecta botones o campos con las acciones del componente. Esto separa lo que la persona ve de las operaciones que actualizan los datos.

Plantilla: feedback y acceso rÃ¡pido — App.vue · líneas 1563–1585

```
1563
1564      <Transition name= "cart-feedback">
1565        <p v-if= "cartFeedback && !isCartOpen && !isCheckoutOpen && !selectedProduct" class="cart-feedback-toast" :class="`cart-feedback-${cartFeedbackTy
pe}` " role="status" aria-live="polite">
1566          <span aria-hidden= "true">{{ cartFeedbackType ==='success'  ?'â œ“'  :'!'  }}</span>
1567          {{ cartFeedback }}
1568          <button class= "cart-feedback-open" type="button" @click="isCartOpen = true">Ver bolsa</button>
1569        </p>
1570      </Transition>
1571
1572      <button
1573        v-if= "cartCount > 0 && !isCartOpen && !isCheckoutOpen"
1574        class= "floating-cart"
1575        type= "button"
1576        :aria-label= "`Abrir bolsa, ${cartCount} ${cartCount === 1 ? 'artÃculo' : 'artÃculos'}`"
1577        @click= "isCartOpen = true"
1578      >
1579        <span class= "floating-cart-icon" aria-hidden="true">â–±</span>
1580        <span>{{ cartLabel }}</span>
1581        <span class= "floating-cart-total">{{ formatPrice(cartTotal) }}</span>
1582      </button>
1583    </div>
1584  </template>
1585
```